Neurocomputing 451 (2021) 231–251

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

An evolving neuro-fuzzy system based on uni-nullneurons with advanced interpretability capabilities

Paulo Vitor de Campos Souza *, Edwin Lughofer

Department of Knowledge-Based Mathematical Systems, Johannes Kepler University Linz, A-4040, Austria

ARTICLE INFO

Article history:
Received 14 September 2020
Revised 23 February 2021
Accepted 18 April 2021
Available online 23 April 2021
Communicated by Zidong Wang

Keywords:
Evolving neuro-fuzzy system (ENFS-Uni0)
Uni-nullneurons
On-line interpretability of fuzzy rules
Degree of rule changes
Incremental feature importance levels
Indicator-based recursive weighted least squares (I-RWLS)

ABSTRACT

This paper proposes a hybrid architecture based on neural networks, fuzzy systems, and n-uninorms for solving pattern classification problems, termed as *ENFS-Uni0* (short for evolving neuro-fuzzy system based on uni-nullneurons). The model can produce knowledge in an on-line (single-pass) and evolving learning context in a particular form of neuro-fuzzy rules representing the dependencies among input features through IF-THEN type relations. The rules antecedents are thereby realized through uni-nullneurons, which are constructed from n-uninorms, leading to the possibility to express both, AND- and OR-connections (and a mixture of these) among the single antecedent parts of a rule (and thus achieving an advanced interpretability aspect of the rules). The neurons' evolution is done through an extended version of an autonomous data partition method (ADPA). On-line interpretation of the timely evolution of rules is addressed by (i) a concept for tracking the degree of changes of the rules over data stream samples, which may indicate experts/operators how much dynamics is in the process and may be used as a structural active learning component to request operator's feedback in the case of significant changes and (ii) a concept for updating feature weights incrementally. These weights express the (possibly changing) impact degrees of features on the classification problem: features with low weights can be seen as unimportant and masked out when showing rules to an expert (→ rule length reduction). The rules' consequents are represented by certainty vectors and are recursively updated by an indicator-based recursive weighted least squares (I-RWLS) approach (one RWLS estimator per class) where the weights are given through the neuron activation levels in order to gain stable local learning. The model proposed in this paper was successfully compared to related hybrid and evolving approaches in the literature for classifying binary and multi-class patterns. The results obtained by the proposed model show an outperformance of the related works in terms of higher accuracy trend lines over time, while offering a high degree of interpretability through coherent neuro-fuzzy rules to solve the classification problems.
© 2021 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Technological advances and the growing evolution of science allow intelligent tools to be made available to solve complex problems, such as pattern recognition, image identification, or interactions between humans and a machine [1]. To this end, intelligent approaches have been gaining ground in resolving various kinds of problems, especially those related to the permanent updating of the models with new data [2,3].

To address problems with high complexity with a wide variety of data, intelligent hybrid models in the form of neuro-fuzzy systems and fuzzy neural networks are highlighted in the scientific literature, see e.g., [4] or in a recent comprehensive survey [5]. Evolving fuzzy neural network and neuro-fuzzy models can act dynamically in solving online problems, mainly due to their ability to adapt to and evolve according to the problem's nature [6,7]. These models are different from other evolving approaches because they extract knowledge of the analyzed problem, mainly in logical IF-THEN relationships. These fuzzy rules can help understand and represent the knowledge in the data, making it expressable to experts on the subject and highly disseminable to those who intend to solve these complex problems [8].

During the last 15 to 20 years, many evolving neuro-fuzzy techniques emerged, basically differing in their way of how to extract the neurons/rules for data streams, how to incrementally update non-linear parameters in the antecedent space as well as in architectural elements such as the types of neuron applied, among others [9]. Recent comprehensive surveys about evolving neuro-

* Corresponding author.
E-mail addresses: paulo.de_campos_souza@jku.at (P.V. de Campos Souza), edwin.lughofer@jku.at (E. Lughofer).

https://doi.org/10.1016/j.neucom.2021.04.065
0925-2312/© 2021 The Author(s). Published by Elsevier B.V.
This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

fuzzy systems (ENFS) published in [10,11] discuss 80–100 various approaches describing their basic properties and characteristics, also in comparison with parametrization issues, computation demands, etc. The common denominator in all of these approaches is (i) their capability to evolve new neurons and/or fuzzy rules on demand and the fly based on new incoming data samples, thus addressing knowledge expansion required in the case of dynamic system changes and (ii) their capability to update the model parameters in an incremental (often recursive) manner to address high-speed streams and meeting real-time modeling demands. Applications of ENFS approaches comprise prediction and identification of dynamic systems [12], electricity forecasting [13], driving fatigue [14], travel time estimation [15], anomaly detection in data centers [16], micro aerial vehicles controller [17], robust evolving controllers [18,19], optical interconnection network [20], software effort estimation [21], health-care and bioinformatics applications [22,23] and on-line data stream mining [24,25], among others, see also [11] for overview tables.

Recently published work used the structure of hybrid neuro-fuzzy models to solve problems related to prosthetic Hand Myoelectric-Based Control using design and tuning of implementation of linear and fuzzy controllers [26] and a simplified fuzzy rule base. The Ge and Jeng [27] model used a self-learning-adaptive thresholds approach to improve the previous work of the same authors [28] to act on a variable time limit to control the generation of hybrid model rules. Genetic algorithm techniques were used in an evolving fuzzy neural network model to solve modeling the liquid level and oxygen content in the industrial coke oven. Another recent model called IT2-GSETSK [29] uses merge and pruning rules to define a compact architecture of rules to solve problems of non-linear systems. Finally, an evolving hybrid model (EBeTS) that acts on fault prognostics was recently proposed, demonstrating the scope and effectiveness of evolving hybrid intelligent models in solving complex and varied problems [30].

In all of the aforementioned methods and those which can be found in recent extensive surveys [10], the central focus so far lied in *precise stream modeling* aspects, that is, an ENFS model is incrementally evolved to best reflect the current predictive trend in the data stream between input features and a target (class label), with the ultimate aim to obtain the highest possible predictive performance on future samples with a best possible trade-off between plasticity to changes (drifts) [31] and stability in parameter learning [32]. Little attention has been paid so far to the interpretability of the ENFS models' aspects to gain a more in-depth insight into the process. The paper in [8] discusses several essential aspects of interpretability in ENFS in a kind of position-oriented manner, but concrete implementations and proper embeddings of most of these in ENFS approaches are missing. Complexity reduction and associated simplicity assurance as one essential interpretability aspect to assure compact rule bases is an exception, as having been addressed in many ENFS approaches, basically due to incremental, single-pass merging concepts [33] based on neuron/rule distance and overlapping criteria (arising over time due to data cluster fusioning effects), see, e.g., [34,25,35]. Furthermore, almost all of the current ENFS approaches employ standard t-norms (or direct multi-dimensional kernels for generalized rules [36,37]) to construct the rules/neurons' antecedent parts. That always leads antecedent connections in the form of 'ANDs' in the rule antecedents (often computationally realized through their product) but does not allow to address OR-connections, which may offer additional interpretable facets for representing classes through rules.

This article intends to propose a new evolving neuro-fuzzy systems approach with the following properties, leading to advanced interpretability aspects:

- Construction of the model components is done through the design of uni-nullneurons which are based on n-uninorms [38], which is capable of acting dynamically as uninorm [39] or nullnorm [40], two traditional fuzzy logic operators with past usage [41]. This way, it can express a mixture of AND- and OR-connections in one neuron/rule. That allows more flexible interpretation capabilities for modeling classes (e.g., an OR can allow a rule to be more widely spread across the respective feature range to represent a single class).
- The extraction of the data clouds based on which the uni-nullneurons are formed is achieved by an autonomous data partition method (ADPA) seeking for density cloud-based regions [42], in which we extend the rule evolution criterion to a supervised one by using the class labels of incoming data stream samples: overlapping class cases can therefore be better handled by evolving new rules also in the case when samples are falling close to an existing cluster, but have different labels as the majority class in the closest cluster.
- A concept for incrementally calculating feature weights according to their importance for the stream classification problem at hand; this is based on an advanced separability criterion for which we provide single-pass update formulas. The feature weights can be used for further interpretable insights (which features are important during which phase of the process/stream) and also for reducing the rule length when showing rules (extracted from the neurons) to experts/operators.
- A concept for tracking the degree of changes in the rules for tracking (significant) timely based changes in the system and requesting operator's/expert's feedback in the case of significant changes (→ structural active learning).
- Updating the neural output weight (=consequent) parameters, which are provided in the form of certainty vectors including a numerical certainty entry for each class in each neuron/rule, by an extended version of recursive weighted least squares [43,7], which integrates neuron activation levels for local learning and an indicator-based scheme for performing class-wise updates of the parameters—thus, it is termed as I-RWLS.

This paper's main innovation is fuzzification processes that are adaptable to new data and, consequently, generate impacts on the knowledge extracted through fuzzy rules. This model's architecture has a new fuzzy logical neuron capable of acting in flexible ways in solving the problem, streamlining the expected responses so that they are more assertive, and enabling a set of fuzzy rules with variability in their construction. Another factor to be highlighted is the definition of neuron's weights in the first layer through a procedure based on each feature's relevance to the classification problem at hand. That means that inputs less relevant to the classification problem will generate neurons with less significant value in the model outputs' impact.

Our approach has been evaluated on binary and multi-class classification data sets from the UCI repository compared to related SoA works. The results showed improved model accuracies (i) when evolving the models on a training set and evaluating them on a separate test set and (ii) when permanently performing ahead predictions on on-line data stream samples (before updating the models with them) and accumulating the accuracies over time based on comparing the predictions with the correct class labels (according to the interleaved-test-and-then train scenario proposed in [44] and widely used in the stream modeling community)—these results will be highlighted in the results section by corresponding plots and tables. Furthermore, some interpretable rule and feature weight examples will be shown, and their evolution and associated degree of change over time.

Table 1
List of abbreviations (left column) and their associated meanings (right column).

Abbreviation	Meaning
ADPA	Autonomous Data Partitioning Method
ALMMo-0	Autonomous Multi-Model Systems Architecture of Zero-Order Type
AUC	Area Under the Curve
BOLASSO (bolasso)	Boostrap-based Least Absolute Shrinkage and Selection Operator
CPU	Core Processing Unit
DDC-FNN	Data Density-based Clustering for regularized Fuzzy Neural Networks
EBeTS	Error Based Evolving Takagi–Sugeno Fuzzy Model
EDA	Empirical Data Analysis
EFNHN	Evolving Fuzzy Neural Hydrocarbon Networks
ENFS	Evolving Neuro-Fuzzy Systems
ENFS-Uni0	Evolving Neuro-Fuzzy System based on Uni-Nullneurons (our approach)
FNN	Fuzzy Neural Network
HPV	Human PapillomaVirus
IT2-GSETSK	Interval Type-2 Generic Self-Evolving Takagi–Sugeno-Kang fuzzy system
I-RWLS	Indicator-based Recursive Weighted Least Squares
IGN	Incremental Gauss–Newton Step
LARS	Least-Angle Regression
LASSO (lasso)	Least Absolute Shrinkage and Selection Operator
LOFO	Leave-One-Feature-Out
LS	Least Squares
RAM	Random Access Memory
ROT	Result of Treatment
RWLS	Recursive Weighted Least Squares
SMOTE	Synthetic Minority Oversampling TEchnique
SoA	State-of-the-Art
SODA	Self-Organized Direction Aware data partitioning algorithm
SPE-AHN	Stochastic Parallel Extreme Artificial Hydrocarbon Networks
SQL	Structured Query Language
SSR	Sum of the Squares of the Residues

Table 2
List of parameters (left column) and their associated meanings (right column).

Parameter	Meaning
$\vec{a}$	fuzzy set activation levels (relevance)
A_i^j	the jth fuzzy set in the (antecedent part of the) ith rule
b_t	the number of bootstrap replications
$\vec{c}_i$	the center coordinates of the ith cloud (neuron)
C	the number of classes
C_k	the number of existing local clouds (modes)
$D_K(x_k), D_K^G$	local density (in current sample x_k), global density
f_Γ	neuron activation function
g, u, β	neutral elements in uninorms, nullnorms and n-uninorms
J_j	separability measure for the jth feature
L, L_{sel}	the number of extracted clouds, the number of selected neurons
λ	regularization parameter in the Lasso regression
N or d	the number of inputs (features)
μ_k	the current data samples average
μ_k^i	the average of samples falling into the ith cloud (mode)
Ω	final classification activation function
p	conditional transformation function, weighted aggregation
ψ	forgetting factor in I-RWLS
$\rho_{k..}$	hit matrix
$\vec{\sigma}_i$	the standard deviations in all dimensions of the ith cloud (neuron)
S_k^i	the support of the ith local cloud at time instance k
S	similarity degree between two rules
U, NU, N^{un}	uninorm, nullnorm and n-uninorm
$\vec{u}$	unique data point locations
$\vec{v}$	consequent certainty vector for all neurons (binary case)
$\vec{v}_j$	consequent certainty vector for the jth neuron, one entry per class (multi-class case)
x_k	the current data sample (in the kth time instance)
ξ	consensus threshold
$\vec{w}$	feature weights
$y_k, \vec{y}_k$	real class label in the kth sample, indicator vector for the kth class
$z(x_i)$	neuron activation levels in sample x_i
Z_{sel}	neuron activation matrix (rows = samples, cols = neurons)

The paper is composed as follows: Tables 1 and 2 shows a list of abbreviations and parameters with their associated meanings as used throughout the whole paper. Section 2 presents the main theoretical concepts about n-uninorms based on which our uni-nullneurons (the essential components in our model) are constructed. In Section 3, the ideas of the evolving fuzzy neural network architecture, the fuzzy neuron, and all aspects related to incremental, evolving training procedures of the components and parameters in our neuro-fuzzy system will be demonstrated. Section 4 demonstrates the two aspects of advanced model interpretability. In Section 5, all the results in comparison with related SoA works and terms of achieved rules (interpretation) will be presented. Finally, Section 6 highlights the conclusions about the study.

2. Basic theoretical concepts for constructing uni-nullneurons

In this section, the essential concepts about n-uninorms needed to understand the construction of the uni-nullneurons will be presented.

2.1. Uninorm and nullnorm

The uninorms [39] and nullnorms [40] are generalizations of *t-norm* and *s-norm* by relaxing restrictions associated with the one variable, that can change between the [0,1] interval. The identity (uninorm) or absorption (nullnorm) element is provided to assume values in the unit interval.

Through this identity element (g), the uninorms spread *t-norm* and *s-norm* by diversifying the value g in the interval between 0 and 1 enabling the alternation between an *s-norm* (g = 0) and *t-norm* (g = 1). The nullnorm absorption (u) uses these same behaviour, but the difference is in a t-norm/s-norm change (u = 0 the model uses t-norm and when u = 1 is apply the s-norm operator). It is important to highlight that two binary operator BO: $[0,1]^2 \leftarrow [0,1]$ which is monotonic, commutative, associative, and similar.

The uninorm (U) used in this paper is expressed as follows [45]:

$$U(x,y,g) = \begin{cases} g\ T(\frac{x}{g}, \frac{y}{g}), if\ y\ \in [0,g] \\ g + (1-g)\ S\ (\frac{x-g}{1-g}, \frac{y-g}{1-g}),\ if\ y\ \in\ (g,1] \end{cases} \quad (1)$$

At the same time, the nullnorm (NU) is expressed as follows [40]:

$$NU(x,y,u) = \begin{cases} uS(\frac{x}{u}, \frac{y}{u}), if\ y\ \in [0,u] \\ u\ +\ (1-u)\ T\ (\frac{x-u}{y-u}, \frac{1-y}{1-u}), if\ y\ \in\ (u,1] \end{cases} \quad (2)$$

where T and S denote an arbitrary t-norm and t-conorm [46], respectively, and where both norms are commutative and associative [47]. Note that, according to the setting of g and u, more an AND- or more an OR-connection can be emphasized: for instance, if g is close to 0, the first case in (1) is hardly met, thus U represents an OR-connection (to a large extent). In this sense, much flexibility is achieved to have AND/OR connections with different intensity degrees of ANDs and ORs within one rule (as the setting may vary from connection to connection within one rule antecedent). This paper, the value of g, will consider only two possible values: 0 or 1. If the value of g is equal to one, it can be called a disjunctive uninorm (U_d) (leading to a pure OR-connection). If its value is zero, it

can also be called a conjunctive uninorm (U_c) (leading to a pure AND-connection).

2.2. N-uninorms

An n-uninorm (proposed by Akella [38]) is an aggregation operator that was introduced as a generalization of uninorms and nullnorms, N_{un}. It was defined to generalize the neutral element concept, allowing uninorms and nullnorms to improve binary operators' flexibility. Fodor et al. [48] and Hu and Zhong-fu [49], define U as disjunctive uninorm with a neutral element $g \in (0,1)$. If the function U (x,1) is continuous except in $x = g$, then there exist a t-norm T and a t-conorm S as presented in Eq. 1 [47].

A specific case of n-uninorm called 2-uninorms is used in this article. Akella explains that a function V: $[0,1]^2 \rightarrow [0,1]$, being a commutative binary function with $g, u, \beta \in [0,1]$ with $0 \leqslant g \leqslant \beta \leqslant u \leqslant 1$and $0 < \beta < 1$. Therefore, $\{g,u\}_\beta$ is the definition of a 2-neutral element of V if it satisfies [38]:

- V(g, x) =x for all x $\leqslant \beta$
- V(u, x) =x for all x $\geqslant \beta$

The 2-neutral element define the binary operator called 2-uninorm, $N^{un} = [0,1]^2 \rightarrow [0,1]$, if the operator is commutative, associative and has $\{g,u\}_\beta$ [47]. The N^{un} operator can act as a uninorm if $g = \beta = u$, just as it can turn into a nullnorm if $g = 0$ and $u = 0$. This behavior of the 2-uninorm operator is flexible relative to its operators and in this paper as $0 < g < \beta < u < 1$, it can written as follows [47]:

$$N^{un}(x,y,\beta,g,u) = \begin{cases} \beta U_1(\frac{x}{\beta},\frac{y}{\beta}), \ \textit{if } x,y \in [0,\beta] \\ \beta + (1-\beta)\, U_2\,(\frac{x-\beta}{1-\beta},\frac{y-\beta}{1-\beta}), \ \textit{if } x,y \in (\beta,1] \end{cases} \tag{3}$$

where U_1 is a uninorm (Eq. 1) with a neural element (identity) = $\frac{g}{\beta}$ and U_2 is a uninorm (Eq. 1) using $(\frac{u-\beta}{1-\beta})$ like as neutral element. N^{un} has the following properties [47]:

- 1-N^{un}(0,1) $\in \{0, \beta, 1\}$
- 2-N^{un}(0,1)=N^{un}(N^{un}(0,1), x) for all $x \in$ [0,1].
- 3-If N^{un}(0,1)=1, then either N^{un}(0,β)=β with U_1, U_2 are U_d or N^{un}(0,β)=0 with $U_1 \in U_c$ and $U_2 \in U_d$.
- 4- If $U_1, U_2 \in U_d$, then N^{un}(0,1)=1.
- 5- $N^{un}(\beta, 1) = N^{un}(g, 1)$ and $N^{un}(\beta, 0) = N^{un}(u, 0)$.

In order to underline the usage of the 2-uninorm structure, Table 3 represents relevant works and in which context these have used it.

2.3. And-neuron and or-neuron

Logical neurons are functional units that combine logical aspects of processing with learning ability through the system of fuzzy rules. They can be seen as multivariable nonlinear transformations between unit hypercubes, or $[0,1] \rightarrow [0,1]^n$ [56]. Thus, neurons *and* and *or* add the values of fuzzy relevance $\mathbf{a} = [a_1, a_2, \ldots, a_3, \ldots a_N]$ initially combining them individually with their weights $\mathbf{w} = [w_1, w_2, \ldots, w_3, \ldots w_N]$, $\mathbf{a}, \mathbf{w} \in [0,1]^n$ to combine these results in the following way [56]:

$$\mathbf{z} = AND(\mathbf{w};\mathbf{a}) = T_{i=1}^{n}(w_i \ s \ a_i) \tag{4}$$

$$\mathbf{z} = OR(\mathbf{w};\mathbf{a}) = S_{i=1}^{n}(w_i \ t \ a_i) \tag{5}$$

where S and s are *s-norm* (typically product is used) and T and t are *t-norm* (typically probabilistic sum is used).

2.4. Unineuron and nullneuron

The unineuron apply the concepts of uninorm [39] to accomplish more simplified operations according to the functions of activation of the fuzzy neurons. Its construction allows the unineuron to use either concepts of a neuron and, or a neuron or. [45] explain relevant concepts about a unineuron. It processing occurs at two levels called L_1 (when the input signals submited to the structure are calculated individually using the weights) and L_2 (responsable to realize a global aggregation to performed on the results of all first-level components). These neurons use t-norms and s-norms to calculated the outputs of these neurons. The main the two simple steps are listed below:

1- each pair (a_i, w_i) is convert into a one value $b_i = \mathbf{h}\ (a_i, w_i)$;

2- calculate the aggregation of the all values $\mathbf{U}\ (b_1, b_2 \ldots b_n)$, where n is the number of inputs.

The function p is applied for change the inputs and respective weights into individual transformed values. p function used in this paper can be described by [57]:

$$p(w,a,g) = wa + \bar{w}g \tag{6}$$

using the weighted aggregation reported above the unineuron can be written as:

$$z = UNI(w;a;g) = U_{i=1}^{n} p(w_i, a_i, g) \tag{7}$$

Similarly, Nullneuron [40] also uses the two steps to turn the elements in a single value. The main difference is based on the substitution of the neutral element to the absorption element. The nullneuron applied in this paper uses the same unineuron [57] concepts to perform the following calculations using Eq. 6:

$$p(w,a,u) = wa + \bar{w}u \tag{8}$$

and nullneurons can be written as [40]:

$$z = NULL(w;a;u) = NU_{i=1}^{n} p(w_i, a_i, u) \tag{9}$$

2.5. Uni-nullneuron

The main characteristic of fuzzy neurons is their adaptability in managing inputs and their respective weights, using fuzzy logical operators. Therefore, the higher the flexibility in these operators' use, the greater the structure's capacity to adapt to different contexts and calculations necessary to solve complex calculations. Neurons that use uninorms and nullnorms adapt to use t-norms and s-norms and therefore allow these structures to act either as and-neurons or as or-neurons. Based on the adaptability of n-uninorms, this paper proposes using a fuzzy neuron based on 2-uninorms to make the fuzzy neuron more flexible to play the role of unineuron or nullneuron. The logical neuron that uses n-uninorms will be called here uni-nullneuron, and its dynamics will be through parametric modification (steered by g, u and β), as described in the subSection 2.2. This new approach works with similar neurons expressed by Eq. 7 and 9 respectively. The uni-nullneuron also has a function p, termed as *conditional transformation*, of the values of the n-uninorm. Consequently, it can define it as being:

$$p(w,a,\beta,g,u) = \begin{cases} wa + \bar{w}\frac{g}{\beta}, \ \textit{if } U_1 \\ wa + \bar{w}\frac{u-\beta}{1-\beta}, \ \textit{if } U_2 \end{cases} \tag{10}$$

$$z = UNI^{NUL}(w,a,\beta,g,u) = N^{un}{}_{i=1}^{n} p(w_i, a_i, \beta, g, u) \tag{11}$$

Table 3
Literature work with 2-uninorm.

Author	Context	Reference
Wang and Liu	Solve the distributivity equations in uni-nullnorms and grouping functions.	[50]
Wang et al.	Work with specific subclass of fuzzy implications.	[47]
Drygaś and Rak	Functional equation of distributivity between aggregation operators.	[51]
Rak	Functional equations of modularity of special class of aggregation operators.	[52]
Zong et al.	Describe the structure of 2-uninorms on the unit interval	[53]
Wang and Qin	Solve the distributivity equations for 2-uninorms over semi-uninorms.	[54]
Zhang et al.	Distributivity of continuous triangular norms.	[55]

As an n-uninorm can some extent as uninorm or nullnorm, the strength of uni-nullneuron is to act like a unineuron, or nullneuron in its process. This type of neuron creates a new level of use for fuzzy logical neurons based on triangular norms, principally due to the already known ability of unineurons and nullneurons to act as *and* or *or* neurons at the same time over varied input dimensions. That affects that a mixture of ANDs and ORs can automatically connect the rule antecedents. Hence, fuzzy neural networks with these neurons can describe dynamically AND and OR connections, developing comprehensive and advanced interpretable rules.

3. Evolving fuzzy neural network based on uni-null-neuron

The model proposed in this paper has three layers. The number of neurons generated in the model's initial layer is solely based on the input data and automatically elicited from it using the evolving cloud-based approach ADPA (see Section 3.2). An incremental separability criterion (presented in Section 3.4) generates the feature weights w used in the construction of the neurons according to (11). In the second layer, fuzzy neurons, capable of being seen as if-then rules with varying antecedent connections (through ANDs and ORs) achieved through uni-nullneurons, are embedded (fuzzy type III logical neurons). This type of structure acts in order to extract knowledge from a data set through fuzzy relationships. They have the primary function to aggregate the neurons generated in the anterior layer of the model. In the last layer of the model, vectors of numerical singletons are used in the model's defuzzification process, representing certainty degrees in all classes for each rule. These degrees are estimated using an indicator-based least-squares criterion (yielding one regression sub-problem for each class)—see Section 3.6. To avoid that, non-relevant neurons are present in the model, a regularization approach based on resampling and LARS is used in the second layer of the model (Section 3.5). In this case, the selected subset of fuzzy neurons acts as the most usual fuzzy rules that assure the model's further compactness.

3.1. Evolving Fuzzy neural network architecture: Structure of variable neurons and activation functions

The evolving fuzzy neural network proposed in this paper addresses a few aspects present in other relevant literary works—e.g., the fuzzification approach has already been covered in Souza et al. [58]. The significant difference from previous works and the model proposed in this paper is the definition of weights based on a new feature weighing technique instead of a random definition of weights (→ much better interpretability as deterministically determined). Another difference is that fuzzy rules are constructed through uni-nullneurons allowing AND- and OR-connections in the antecedents in the second layer. Finally, the weights connect the second layer of the model with the output layer through an indicator-based recursive weighted least-squares approach (one RWLS estimator per class).

The first layer contains L neurons whose activation functions are membership functions of fuzzy sets generate by the density of the input data. For each input featurex_{ij}, L clouds are defined A_{lj}, $l = 1, \ldots, L$ where membership functions are the activation functions of the corresponding neurons. Therefore, the final results of the fuzzification layer are the membership degrees (a) connected with the input values, i.e., $a_{jl} = \mu_l^A$ for $j = 1 \ldots N$ and $l = 1 \ldots L$, where N is the number of inputs and L is the number of fuzzy sets for each input, automatically resulting from the ADPA algorithm (as described in the next section).

The layer responsible for building fuzzy rules is composed of L fuzzy logic neurons called Uni-nullneuron. Each fuzzy logical neuron performs a weighted aggregation of all of the first layer outputs. This aggregation is performed using feature weights w_{il} (for $i = 1 \ldots N$ and $l = 1 \ldots L$), which depend on the importance of features according to their discriminatory power among the classes in the data set. They are obtained through an incremental approach to calculate and update the feature weight vector due to an advanced separability criterion—see Section 3.4. For each input variable j, only one first layer output a_{jl} is defined as input of the l-th neuron. These neurons can extract knowledge from the database and generate fuzzy rules.

The process carried out in the third layer is also seen as defuzzification process with an aggregation neural network composed of a single neuron.

$$y = \Omega\left(\sum_{j=0}^{l} f_\Gamma(z_j, v_j)\right) \tag{12}$$

where $z_0 = 1$, v_0 is the bias, and z_j and v_j, $j = 1, \ldots, l$ are the output of each fuzzy neuron of the second layer and their corresponding weight, respectively and f_Γ represents the neuron activation function. The Ω function acts similarly to the sign function in binary pattern problems, which returns value 1 if the analyzed number is positive and -1 if it is negative. Thus, the Ωfunction is described as:

$$\Omega = \begin{cases} 1, \ if \ \sum_{j=0}^{l} f_\Gamma(z_j, v_j) > 0 \\ -1, \ if \ \sum_{j=0}^{l} f_\Gamma(z_j, v_j) < 0 \end{cases} \tag{13}$$

Likewise, when the model acts in solving problems with multiple classification outputs, $\sum_{j=0}^{l} f_\Gamma(z_j, \vec{v}_j)$ delivers an output vector $\vec{o}$ (as v_j turns into a vector of outputs $\vec{v}_j$ for the jth neuron), where each entry is the overall certainty (among all neurons/rules) that the sample belongs to the corresponding class. Therefore, when the model has C class outputs, Ω is given by:

$$\Omega = argmax_{i=1,\ldots,C} o_i \tag{14}$$

Fuzzy rules can be extracted from the network topology in the form as presented in Eq. 15 below:

$$
\begin{aligned}
&Rule_1: \ If\ x_1\ is\ A_1^1\ with\ impact\ w_{11}\ldots\\
&and/or_{(g,u,\beta)}\ x_2\ is\ A_1^2\ with\ impact\ w_{21}\ldots\\
&Then\ y_1\ is\ [v_{11}\ldots v_{1C}]\\
&Rule_2: \ If\ x_1\ is\ A_2^1\ with\ impact\ w_{12}\ldots\\
&and/or_{(g,u,\beta)}\ x_2\ is\ A_2^2\ with\ impact\ w_{22}\ldots\\
&Then\ y_2\ is\ [v_{21}\ldots v_{2C}]\\
&\ldots.Rule_L: \ If\ x_1\ is\ A_L^1\ with\ impact\ w_{1L}\ldots\\
&and/or_{(g,u,\beta)}\ x_2\ is\ A_L^2\ with\ impact\ w_{2L}\ldots\\
&Then\ y_L\ is\ [v_{L1}\ldots v_{LC}]
\end{aligned}
\tag{15}
$$

where C is the number of classes in the problem. $\vec{v}_i = [v_{i1}, \ldots, v_{iC}]$ can be seen as a certainty vector, where each entry gives rise about the certainty of the rule to the corresponding output class, i.e., that the corresponding class should be responded (when a sample falls in the region of the rule). Hence, our rules can represent certainty degrees in the rule outputs, which provide a higher level of interpretability, especially in terms of the degree of class overlap in the local region the rule represents. For instance, a vector of $[0, 0.2, 0.8]$ denotes a high certainty in Class #3, a low one in Class #2, and no certainty at all in Class #1. Such a rule tends to represent an output Class #3, but with some overlap to Class #2. In this way, the concept of (degree of) conflict [59] can be nicely represented.

In the special case of binary classification, $v_1, \ldots, v_L$ become singleton consequents representing the classification response, and the connectives between the rules are defined by the n-uninorm used. These values are in $[-1, 1]$, where a value close to -1 represent a high certainty in Class 0 (and a low one in Class 1) and a value close to 1 vice versa; a value of 0 represents full uncertainty. w_{il} represents the weight of feature i in the lth rule and impacts the activation level of the uni-nullneuron according to (11).

The values of g, u, and β in the AND/OR connections of the rule antecedents define how the rules can be finally read. In particular, this should be seen in accordance to Table 4, where in the second column it is highlighted which norm is induced as fuzzy operator with all possible combinations of the values g, u and β, when assuming that these take either 0 or 1—in the case of s-norm, the connection of two antecedents can be read as 'OR', in the case of t-norm, the connection can be read as 'AND'.

Fig. 1 illustrates the general feedforward topology of the fuzzy neural networks used in this paper. It presents the ADPA technique's structure in the evolution and adaptation of the neurons in the first layer, constructing the fuzzy neurons in the second layer, and the aggregation neuron in the third layer. The first two layers form a new neuro-fuzzy inference system that can create fuzzy rules that extract knowledge from the analyzed problem.

The output neural network's weight obtains the fuzzy inference system conclusions through indicator based least squares concepts (one LS estimator per class). Therefore, the first two layers' extracted rules results describe the relevance of a mixture of the problem features to concern the expected solutions.

3.2. Autonomous Data Partitioning—ADPA

For extracting the correct number of neurons and their (center and width) parameters autonomously from data, we employ the Autonomous Data Partitioning (ADPA) method, proposed by Gu

Table 4
Relationship between g, u and β and the respective fuzzy operators.

g	u	β	Fuzzy Operator	Fuzzy Neuron
0	0	0	s-norm (OR)	unineuron
0	0	1	s-norm (OR)	nullneuron
0	1	0	t-norm (AND)	andneuron
0	1	1	s-norm (OR)	orneuron
1	0	0	s-norm (OR)	orneuron
1	0	1	t-norm (AND)	andneuron
1	1	0	t-norm (AND)	unineuron
1	1	1	t-norm (AND)	unineuron

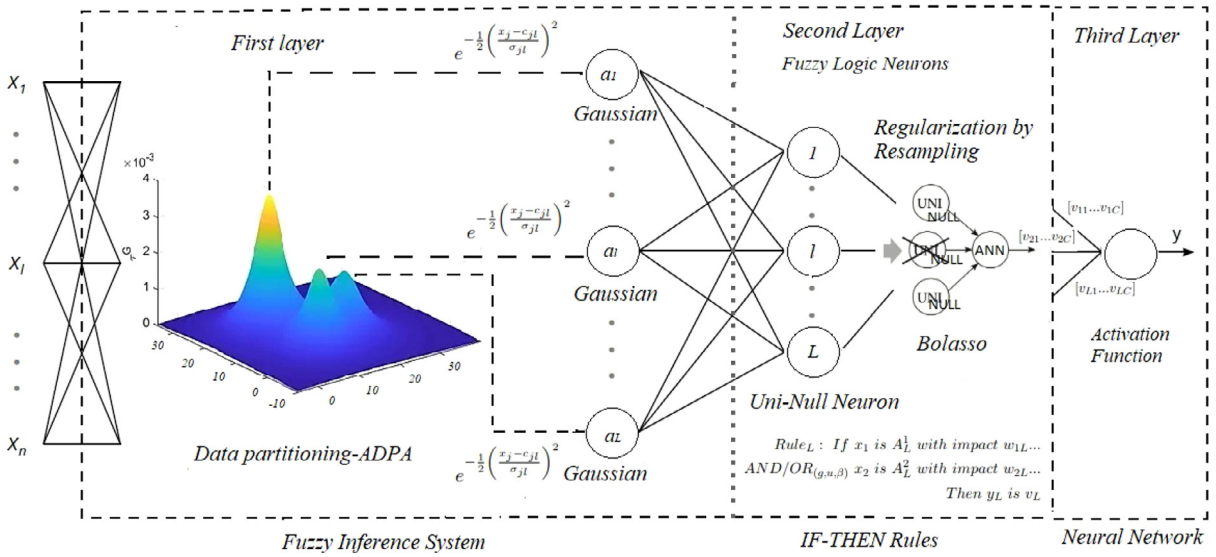


Fig. 1. FNN architecture.

et al. [42]. This technique's main advantage is to use local mode to partition the input space in structures called data clouds, using approaches similar to the Voronoi tessellation technique. Its main advantage is the objective representation of the actual distribution of data, creating clusters in arbitrary shape, opposing traditional clustering techniques that inductively represent the space of characteristics. The empirical evaluation of the proposed fuzzification technique uses data density concepts, allowing the organization and essence of the data to determine the number of clusters needed to solve it autonomously.

The technique of autonomous clustering is supported by the approach proposed in [60], called Empirical Data Analysis (EDA) framework. The works of [61,62] broadly discuss the concepts and practices that the technique can perform for data clustering analysis. In a nutshell, EDA is an approach based entirely on empirical observations of discrete data points and their mutual position. They obtain a unique pattern in the data space by evaluating four parameters recursively calculated: the cumulative proximity, the standardized eccentricity, the density, and the multimodal typicity of the data evaluated in the problem [61].

EDA considers the input variables of the intelligent model as $\mathbf{x} = \{x_1, x_2, \ldots, x_k\} \in \mathbb{R}^d$, where the indices k indicate the time instance at which the data point arrives. We can define the set of unique data point locations in time instance k as $\mathbf{u}= \{u_1, u_2, \ldots, u_L,\} \in \mathbb{R}^d$ and the corresponding number of times that $f_1, f_2, \ldots, f_L$ different data points occupy the same unique locations. Based on u_1, u_2, …, u_L, and $f_1, f_2, \ldots, f_L$, it is possible to reconstruct the data set $x_1, x_2, \ldots, x_k$ exactly if necessary, regardless of the order of arrival of the data points [61]. The ways of determining the distance, proximity, and formation of data clouds involve the concepts of cumulative proximity (π_k^u) described in [60] as the measure that determines how close an evaluated sample is to a group of existing data in the analysis of the problem. Mathematically, we can write it as (16), where the distance (usually of the Euclidian or Mahalanobis type) between two single sites is defined by $d(u_i, u_j)$[61]:

$$\pi_k^u(u_i) = \sum_{j=1}^{L} d^2(u_i, u_j), i = 1, 2, \ldots, L \tag{16}$$

where L is the number of the unique data samples. Likewise, local density is defined as (17) [61].

$$D_k(x_k) = \frac{\sum_{j=1}^{k}\sum_{l=1}^{k} d^2(x_j, x_l)}{2k\sum_{l=1}^{k} d^2(x_k, x_l)} \tag{17}$$

where $\sum_{l=1}^{K} d^2(x_k, x_l) = \sum_{l=1}^{K} \|x_k - x_l\|^2$ and $\sum_{j=1}^{K}\sum_{l=1}^{K} d^2(x_j, x_l) = \sum_{j=1}^{K}\sum_{l=1}^{K} \|x_j - x_l\|^2$. It is can be simplified using the average of x_K, $\mu_K$and the average scalar product, $X_K$as [42]:

$$um_{l=1}^{K}\|x_k - x_l\|^2 = K\left(\|x_k - \boldsymbol{\mu}_K\|^2 + X_K - \|\boldsymbol{\mu}_K\|^2\right) \tag{18}$$

$$\sum_{j=1}^{K}\sum_{l=1}^{K}\|x_j - x_l\|^2 = 2K^2\left(X_K - \|\boldsymbol{\mu}_K\|^2\right) \tag{19}$$

And for recursively updating parameters, they are expressed as [42]:

$$\boldsymbol{\mu}_k = \tfrac{k-1}{k}\boldsymbol{\mu}_{k-1} + \tfrac{1}{k}x_k; \quad \boldsymbol{\mu}_1 = x_1; \quad k = 1, 2, \ldots, K \tag{20}$$

$$X_k = \frac{k-1}{k}X_{k-1} + \frac{1}{k}\|x_k\|^2; X_1 = \|x_1\|^2; k = 1, 2, \ldots, K \tag{21}$$

Therefore, local density can be defined recursively by the expression [42]:

$$D_K(x_k) = \frac{1}{1 + \frac{\|x_k - \boldsymbol{\mu}_K\|^2}{X_K - \|\boldsymbol{\mu}_K\|^2}} \tag{22}$$

The EDA measurements are responsible for operating the ADPA algorithm's functions by extracting the observed data set properties, releasing it from the need to make available previous data about the data generation model and specific parameters [42].

For the use of the ADPA algorithm, it is necessary to identify in the data the weighted sum of the local density by a similar occurrence in $f_1, f_2, \ldots, f_{LK}$. This factor is called global density (D_K^G), and it is defined for unique data samples together with their corresponding numbers of repetitions in the dataset/stream. It is expressed in (23).

$$D_K^G(u_k) = \frac{f_k}{\sum_{j=1}^{L_K} f_j} D_K(u_k) = \frac{f_k}{1 + \frac{\|u_k - \boldsymbol{\mu}_K\|^2}{X_K - \|\boldsymbol{\mu}_K\|^2}} \tag{23}$$

The ADPA method has two approaches that can be used to construct the data pool in the cloud. In this paper, we use the evolving approach, initiating its approach with a single sample, thus allowing the model to start its procedure without prior knowledge of the data and to autonomously evolve the clouds in order to suit the nature of the new data submitted to the problem [42].

The first step in this approach is defining a problem samples order concerning the data's overall model. That is done by organizing the exclusive data $\{u\}_K$into an indexed list $\{u^*\}_K$ based on their global density and the samples' mutual distance. To do this, all global densities of exclusive data are calculated so that the data sample with the highest global density value is the first reference value in this indexing list. It is defined by [42]:

$$u^{*1} = \arg\max_{j=1,2,\ldots,L_K}\left(D_K^G(u_j)\right) \tag{24}$$

According to Gu et al. [42], this element is then removed from the original list and indexed as the first in the index list. The second value to be indexed is the closest to the reference value. Thus, this procedure occurs until all the analyzed values are adequately indexed.

The second step of the ADPA approach is to detect the local maxima in the indexed list ($\{u^*\}_{L_K}$) using the ranked global density ($\{D_K^G(u^*)\}$)—please refer to [42] for formulas and conditions.

The third stage consists of the formation of data clouds. The maximum location ($\{u^{**}\}_{L_K^*}$) of the index list is used to attract samples ($\{x\}_K$) to them according to the nearest distance among all locations (winner takes all). The assignment of data samples to the local maxima allows the formation of a data cloud based on Voronoi tessellations around them.

The last stage of the initial ADPA approach is to filter local modes, allowing less representative clouds without being combined with others of equal similarity, thus forming a more robust and representative cloud partitioning. Finally, only cloud centers with maximum global–local densities indicated by $\{\boldsymbol{\mu}^*\}_{L_K^{**}} = \{\boldsymbol{\mu}^{*j} \mid j = 1, 2, \ldots, L_K^{**}, L_K^{**} \leqslant L_K^*\}$ are obtained. They are then used as local modes to form data clouds at stage 3 and are filtered at stage 4 [42]. The last two steps of ADPA are repeated until all distances between existing local modes exceed $\frac{\gamma_K^c}{2}$ and L clouds remain. Finally, we obtain the remaining centers with the local maximums and use them as local modes to form clouds of data. The formation includes the elicitation of their centers $\vec{c}_i, i = 1, \ldots, L$, standard deviations among all features $\vec{\sigma}_i, i = 1, \ldots, L$ and supports $k_i, i = 1, \ldots, L$ (i.e., how many samples

are falling closest to each cluster) based on the Voronoi tessellations [42]. Centers and standard deviations are then directly used as parameters to construct our Gaussian neurons.

The evolving approach is summarized in three main steps [42]:

- **First step: Initialization.** It works with the structure defined in offline mode, considering each new sample for the process of structure evolution, updating fundamental parameters in the creation of clouds [42].
- **Second step: Update and check rule evolution condition.** For each sample of data newly arrived at the current time instance, denoted as x_k, the global meta–parameters (especially, the density of the new data samples and of the centers) of the problem are updated with x_k recursively by (23) (using (20) and (21)), using C_k as the number of existing local modes at the kth time instance [42]. Therefore, there are consequently updates on the centers, sigmas, and the local densities of the previously defined centers with each new incoming sample.
  Whether the new sample can form a new data cloud or belong to an existing one, is decided by the following equation:

$$IF\left(D_k(x_k) > max_{i=1}^{C_k}(D_k(\boldsymbol{\mu}_k^i))\right) OR \left(D_k(x_k) < min_{i=1}^{C_k}(D_k(\boldsymbol{\mu}_k^i))\right) \\ THEN(x_k \text{ becomes a new focal point}) \tag{25}$$

  If the condition expressed in the equation is met, a new cloud is created with $x_k$being its local mode, that is,

$$C_k \leftarrow C_k + 1 \ \boldsymbol{\mu}_k^{C_k} \leftarrow x_k \ S_k^{C_k} \leftarrow 1 \tag{26}$$

  The motivation of this condition follows those proposed ones in Gu et al. [42] before, which triggers the evolution of a new cloud whenever the global density of the current sample exceeds the global density of all existing centers: this means that a new sample falls into a dense region where no cloud has been formed so far (otherwise a near cloud would have higher density). It also triggers the evolution of a new cloud whenever the current sample's global density is very low, i.e., lower than all existing clusters. That means that it falls into an (arising) unexplored region representing novelty content in the data.
- **Third step: check cloud assignment condition.** On the other hand, if the condition is not met, the cloud closest to $x_k(\boldsymbol{\mu}_k^n)$ is found. Thereby, the following condition is checked to decide whether $x_k$is assigned to an already existing data cloud or not:

$$IF\left(\|x_k - \boldsymbol{\mu}_k^n\| \leqslant \frac{\eta_k^c}{2}\right) THEN(x_k \text{ is assigned to } \boldsymbol{\mu}_k^n) \tag{27}$$

  If this condiction is not met, a new cloud is evolved as well according to (26).

The cloud values identified in the approach will serve to construct fuzzy logical neurons according to the uni-nullneuron concept discussed in Section 2.5. These neurons make up the first layer of the model, and the definition of its synaptic weight will be given in the following subsection.

3.3. Extended version of ADPA (supervised rule evolution)

The change that this paper proposes to the fuzzification method proposed by Gu et al. [42] is related to the use of the target labels in order to transform the unsupervised approach into a supervised one. The essential motivation stems from the fact that samples can occur close to an existing cloud. Thus condition (27) is met, but may have a different class label than the cloud represents (i.e., than the majority class present in that cloud). This situation indicates an (arising) class overlap with new incoming samples in that region. Thus, a new cloud should be evolved to represent the other (or new) class (and thus omitting the formation of a large cloud containing mixed classes to a similar extent, which would not provide any meaningful neuron/rule knowledge representation the classes in that region).

Our idea is to introduce a comparator term in (27), seeking to verify if the output obtained by the fuzzification method corresponds to the expected label of the evaluated class:

$$IF\left(\|x_k - \boldsymbol{\mu}_k^n\| \leqslant \frac{\eta_k^c}{2}\right) AND \ (argmax(\rho) == y_k \ and \ v \geqslant 0.80) \\ THEN(x_k \text{is assigned to} \boldsymbol{\mu}_k^n) \tag{28}$$

where y_k is the real label of the k-th stream sample x_k and ρ represents a matrix, where the (i,j)-th entry denotes the number of samples falling into cloud i (i.e., for which cloud i is nearest) and belonging to class j. Furthermore, v denotes the frequency of the majority class in the closest cloud (k) compared to those of the other classes and is thus calculated by:

$$v = \frac{max(\rho_{k,.})}{\sum_{j=1}^{C}\rho_{k,j}}$$

with C the number of classes. So, only when the label of the new sample matches the label of the majority class in the nearest cloud k, and denotes no significant class overlap (as a percentage of samples from the majority class is still above 80), the new sample is assigned to its closest cloud. Otherwise, a new cloud is always evolved (due to at least 20 percent class overlap) according to (26).

3.4. On-line incremental feature weight calculation

Traditionally, the definition of neuron weights w in the first layer of fuzzy neural networks occurs through the use of fuzzy clustering techniques and the construction of these neurons according to (11), accompanied by synaptic weights defined at random, as can be seen in previous works by de Campos Souza et al. [63,64,21,65,58]. However, despite the ease of constructing this approach, it does not favor the interpretability of fuzzy rules, since the evaluation of the feature for the problem can have significant impacts or no relevance to the problem. This factor also makes it increasingly random, non-deterministic, and hard to interpret (obtaining different neurons and rules for different run-throughs with the same data). To solve this problem, we implement the definition of feature weights through the online incremental weighing concept proposed in [66].

The proposed approach to the definition of weights based on online training seeks to reduce the problem's dimensionality in a gradual evolving manner. Allowing the weights relative to the features evaluated by the fuzzy neural network varies according to the momentary relevance of the samples evaluated by the problem. The proposed technique in [66] seeks to permanently update the feature weights with new incoming stream samples to reflect the fundamental importance of the features for the stream classification problem.

Continuous weights can be seen as levels of importance of features in the first layer of the model: their impact on discriminating two or more classes or identifying the most relevant dimensions to act on classification problems. Whenever feature weights approach 0, their importance is relatively low compared to others with values close to 1. In this sense, they can also be ignored when showing rules to the operators/experts, see also Section 4. However, unlike feature selection, feature weighting's main advantage is notorious in the learning scenario online. Incremental feature selection abruptly changes weights from 0 to 1 or vice versa. It requires changes to the classifier input structure that are difficult or almost

impossible to compensate in a smooth continuous during online learning mode without forcing a time-intensive re-training on a more significant portion of batch samples seen so far [66].

The basic idea is to extract feature weights based on the usage of a class separability criterion, which in [66] is defined based on the Dy-Brodley separability criterion (a robust extension of Fisher's separability criterion) by:

$$J = \delta(S_w^{-1} S_b) \tag{30}$$

where $\delta(S_w^{-1} S_b)$ defines the sum of the diagonal elements of the matrix $S_w^{-1} S_b$ [66]. S_b denotes the dispersion matrix between classes that measure the class's dispersion averages to the total average, and S_w denotes the within scattering matrix that measures the samples' dispersion in their class averages. The latter can be calculated through the average of the class-wise covariance matrices for exact formulas, see [66]. The higher J becomes, the better is the separability of the classes.

The idea is then to use the criterion in (30) for estimating the importance of each feature for discriminating among the classes. Thereby, in [66], two variants are suggested: single feature-wise calculation and leave-one-feature-out (LOFO) approach. The former uses each feature (and only this one) one time and calculates S_w^{-1} and S_b and finally (30) for each feature $\rightarrow J_1, \ldots, J_N$ for N features in the data set. The latter discards each feature from the complete set and then calculates (30) for each obtained subset $\rightarrow$ again obtaining $J_1, \ldots, J_N$ for N features in the data set.

In the second case, the lower the value of J_i becomes, the more important feature i is because feature i was discarded. Thus, and aiming for a relative importance among all features (relative to the most important feature, which should receive a weight of 1), the feature weights are assigned by:

$$w_j = 1 - \frac{J_j - \min\limits_{i=1,\ldots,N} J_i}{\max\limits_{i=1,\ldots,N} J_i} \tag{31}$$

That guarantees that the feature causing the minimal value of J_j (hence most important one) gets a weight of 1, whereas all others are relatively compared to the minimum: when they are close, the weight will also be almost 1, in the case when they are significantly higher, the weight will be forced towards 0.

In the first case, the higher the value of J_i becomes the more important feature i is for discriminating among the classes. Thus, feature weights are assigned by:

$$w_j = \frac{J_j}{\max\limits_{i=1,\ldots,N} J_i} \tag{32}$$

The separability criterion's incremental update acts on the matrices represented in S_w and S_b. These matrices can be updated during the incremental mode uniquely, see [66] for details.

3.5. Regularization method based on replications

A universal algorithm used for estimating the parameters of a regression model and selecting relevant characteristics is the Least Angle Regression (LARS) [67]. LARS is a regression algorithm for high-dimensional data that is capable of estimating not only regression coefficients but also a subset of candidate regressors to be included in the final model. A modification of the LARS allows the creation of the lasso using the ordinary least squares, a restriction of the sum of the regression coefficients [67]. Consider a set of n distinct samples (x_i, y_i), where $x_i = [x_{i1}, x_{i2}, \ldots, x_{iN}] \in \mathbb{R}^N$ and $y_i \in \mathbb{R}$, the cost function of Lasso algorithm can be defined as:

$$\sum_{i=1}^{K} \|z(x_i)\mathbf{v} - y_i\|_2 + \lambda \|\mathbf{v}\|_1 \tag{33}$$

where λ is a regularization parameter, commonly estimated by cross-validation, and $z(x_i)$ the activation levels of all neurons in the sample x_i. The first term of (33) writes to the sum of the squares of the residues (SSR). The second term is an L_1 regularization term. It is responsible for decreasing as many coefficients as possible to (ward) 0 (as it measures the norm of the coefficients). Thus, (33) achieves a dimensionality reduction in the output weight space, ideally yielding a sparse solution [67].

In this sense, the LARS algorithm can be employed to implement the selection of the most informative neurons, as the contribution of each neuron is associated with its output weight(s) v.: the weight of 0 (or close to 0) means that the neuron does not (hardly) contribute to the final output according to a linearly weighted output conducted through f_Γ; thus it can be deleted during final model construction.

Bolasso can be seen as a regime of consensus combinations where the most significant subset of variables obtained on various bootstrap replications is supported [68]. Fig. 2 presents an example of how the approach selects the best neurons through resampling and statistical techniques. Resampling helps to improve the robustness of neuron selection: if the output weights of neurons are outweighed toward 0 based on the execution of LARS on several bootstraps, the omission of these can be seen as more significant than in the case of using one single batch set. Thus, selection has to be confirmed among more bootstraps such that a neuron is finally active in the model: this is represented by the 'intersection' of output weights during the model selection stage in Fig. 2.

3.6. Update of the fuzzy neural network weights based on Recursive Least Squares

First, the initial weights for the finally selected neurons (those according to Bolasso selection discussed above) are estimated from the initial data set through the Moore–Penrose pseudo inversion technique as also used in [69] (there for binary classification problems), which we extend here to an indicator-based version in order

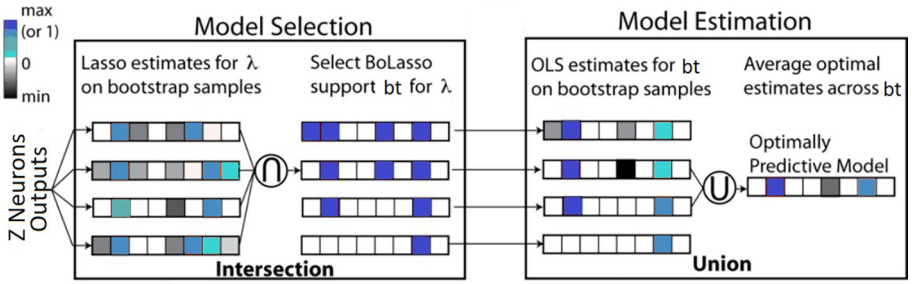


Fig. 2. Example of neuron selection using Bolasso.

to being able to apply it to a multi-class classification problem with non-ordered classes (default case). This leads to the estimation of a vector $\vec{v}_k$ per class separately through:

$$\vec{v}_k = Z_{sel}^{+}\vec{y}_k \forall k = 1,\ldots,C \tag{34}$$

where c denotes the number of classes and $Z_{sel}^{+} = Z_{sel}^{T}Z_{sel}$ is the pseudo-inverse of the Moore–Penrose matrix [70] of Z_{sel}. Z_{sel} contains the activation levels (as rows) for all selected neurons (as columns). y_k denotes the column indicator vector containing 1s at the row positions for samples belonging to class k and 0 for all samples belonging not to class k. This leads to an indicator-based least squares estimation (regression problem on $\{0,1\}$), where a different output weight vector $\vec{v}_k$ (over all selected neurons) per class is achieved. That is, following the well-known one-versus-rest classification realized through regression by indicator matrix [71], which we extend here to a non-linear version according to the integration of the activation levels in the regression matrix Z_{sel}. According to the analysis in [72], the non-linear version does not suffer from any class masking effects (as is the case for the classical linear version, see [71]).

For new incoming on-line stream samples, the weights are recursively updated to meet the system dynamics. Therefore, we employ the recursive weighted least squares (RWLS) approach [73], which is a fast converging well-known estimator, as an incremental Gauss–Newton step is embedded which can converge to the real optimum within single iterations (thus, single sample updates). We extend RWLS to an indicator-based version for each class, termed as I-RWLS, with formulas for updating the $\vec{v}_k$ column vector for the kth class given by (with an update from sample $t-1$ to t):

$$\eta = \vec{z}^{t}Q^{t-1}\left(\psi + (\vec{z}^{t})^{T}Q^{t-1}\vec{z}^{t}\right)^{-1} \tag{35}$$

$$Q^{t} = (\mathrm{I}_{L^t} - \eta^{T}\vec{z}^{t})\psi^{-1}Q^{t-1} \tag{36}$$

$$\vec{v}_k^{t} = \vec{v}_k^{t-1} + \eta^{T}(y_k^{t} - \vec{z}^{t}\,\vec{v}_k^{t-1}) \tag{37}$$

where the index k again denotes the class index $k = 1,\ldots,C$. $\vec{z}^{t}$ denotes the regressor (row) vector of the current sample (i.e. the activation levels of all neurons in the current stream sample), η is the current kalman gain (row) vector, $I_{L_s^t}$ is an identity matrix based on the number of neurons in the second layer, $L_s^t \times L_s^t; \psi \in]0,1]$ denotes a possible forgetting factor, but is to 1 per default (no forgetting). Q denotes the inverse Hessian matrix $Q = (Z_{sel}^{T}Z_{sel})^{-1}$ and is set initially as $\omega I_{L_s^t}$, where ω a big number (e.g., 1000), such as in Rosa et al. [43], see also Chapter 2 in [74] for a detailed convergence analysis.

The whole learning algorithm can be synthesized, as described in Algorithm 1. It has two parameters: 1.) the number of bootstrap replications, b_t; 2.) the consensus threshold, ξ. These are both related to the initial (off-line) training part and can be optimized through cross-validation etc. It is remarkable that for on-line training, no parameter needs to be tuned explicitly, leading to a parameter-free on-line learning method, whose pseudo-code is presented in Algorithm 1.

Algorithm 1: ENFS-Uni0 Training and Update Algorithm

Initial Batch Learning Phase (Input: data matrix X with K samples):

(1) Define bootstrap replications, b_t.
(2) Define a consensus threshold, ξ
(3) Extract L clouds in the first layer using the ADPA approach (L is automatically estimated therein).
(4) Estimate center values $\vec{c}$ and widths $\vec{\sigma}$ for the L clouds derived from ADPA.
(5) Calculate the combination (feature) weights $\vec{w}$ for neuron construction using (31) or (32).
(6) Construct L logic neurons on the second layer of the network by welding the L fuzzy neurons of the first layer, using uni-nullnorms concept according to (11) and the centers $\vec{c}$ and widths $\vec{\sigma}$.
(7) **for** $i = 1,\ldots,K$ **do**
(7.1) Calculate the regression vector $z(x_i)$.
(7.2) Store it as one row entry into the activation level matrix Z.
end for
(8) Select significant L_s neurons using Bolasso method according to b_t and ξ.
(9) Extract reduced activation level matrix Z_{sel} according to the L_s selected neurons.
(10) Estimate the weights of the output layer for all classes $k = 1,\ldots,C$ by (34) using Z_{sel} and indicator vectors $\vec{y}_k$.

Update Phase (Input: single data sample $\vec{x}_t$):

halfline

(11) Update L_s clouds and evolving new one on demand (due to rule evolution conditions (26) and (28)) in the first layer using extended evolving ADPA approach ($\rightarrow L_{s,upd}$ clouds).
(12) Update the feature weights $\vec{w}$ by updating the within- and between-class scatter matrix and recalculating (31) or (32).
(13) Perform Steps (4) and (6).
(14) Calculate the degree of change of all neurons by (38).
(15) Calculate the regression vector $z(\vec{x}_t)$.
(16) Update the weights of the output layer by (37).

Additionally, Fig. 3 shows the main steps performed by the model proposed in this paper in the off-line and evolving phases.

4. Advanced interpretation of the rules extracted from the evolving neurons

4.1. Measuring the degree of rule changes over time

Apart from the linguistically readable IF-THEN structure, which can be directly extracted from the generated clouds (through the network topology) as demonstrated in Section 3.1, another form of interpretability in a data stream modeling, evolving sense would be to track the degree of changes of the rules over time (thus, a timely-based interpretation criterion). That may be an essential indicator within an interactive environment for users/experts to tell them whether and when to re-inspect changed rules (and receive some feedback upon it): only in the case of significant changes is it worth doing so (to decrease interaction effort). In this sense, the degree of change of rules could be used as a *structural active learning based* trigger for requesting expert's feedback, indicating whether the knowledge (and background) of rules have significantly changed during the adaptation new data samples.

In order to meet on-line and especially single-pass demands, we propose the usage of a fast geometric-based criterion to measure the degree of change of a rule, which does not require any past data samples. Therefore, we lean on the idea in [75] where the intersection degree of Gaussian membership functions (note that we extract Gaussians from the clouds to form the FNN topology) is used as an indicator whether two (different) rules have become overlapping or not (which can happen over data stream samples due to cluster fusion effects). In particular, inspecting the overlap

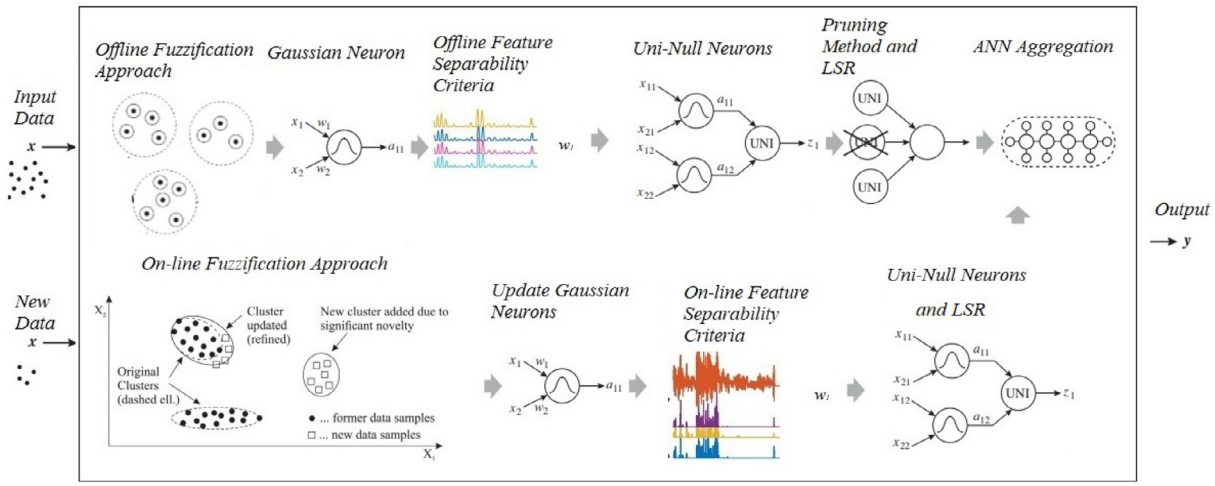


Fig. 3. Steps performed in the ENFS-Uni0 model—off-line (upper row) and evolving phase, where our approach updates the clouds (clusters) in the 'on-line fuzzification step', the Gaussian neurons and the feature weights, all incrementally in a single-pass mode based on new incoming data samples; afterwards, the neurons are constructed based on the updated components.

degree of a rule i before and after its update (termed as $R_i(bef)$ and $R_i(after)$) dimension-wise and calculating an amalgamated value can be used similarity degree $S(R_i(bef), R_i(after))$. The degree of change is then given by

$$\Lambda(R_i) = 1 - S(R_i(bef), R_i(after)) \tag{38}$$

$bef = N - n$ and $after = N$assuming that n new samples have passed the data stream-based adaptation phase with Nsamples used so far for model training and adaptation.

The dimension-wise calculation of S is justified by the fact that the antecedents of fuzzy rules are always represented by a conjunction of single dimension-wise membership functions. Therefore, the approach is per the viewpoint of a fuzzy rule base: two rules are only similar if all their antecedent parts are similar. The x-coordinates of the intersection points of two Gaussians used as fuzzy sets in the same antecedent part of Rule i (here for the jth) before and after its update can be calculated by [75]:

$$inter_x(1,2) = -\frac{c_{bef,j}\sigma^2_{after,j} - c_{after,j}\sigma^2_{bef,j}}{\sigma^2_{bef,j} - \sigma^2_{after,j}} + - \sqrt{\left(\frac{c_{bef,j}\sigma^2_{bef,j} - c_{bef,j}\sigma^2_{after,j}}{\sigma^2_{after,j} - \sigma^2_{bef,j}}\right)^2 - \frac{c^2_{bef,j}\sigma^2_{after,j} - c^2_{after,j}\sigma^2_{bef,j}}{\sigma^2_{after,j} - \sigma^2_{bef,j}}} \tag{39}$$

with $c_{bef,j}$ the jth center coordinate of the rule *before* its update, $c_{after,j}$ the jth center coordinate of the rule *after* its update.

The maximal membership degree of the two Gaussian membership functions in the intersection coordinates is then used as overlap and thus similarity degree of the corresponding rules' antecedent parts in the jth dimension:

$$S_{bef,after}(j) = \max(\mu_i(inter_x(1)), \mu_i(inter_x(2)))$$

with $\mu_i(inter_x(1))$ the membership degree of the jth fuzzy set in Rule i in the intersection point $inter_x(1)$. The amalgamation overall rule antecedent parts leads to the final similarity degree between the rule before and after its update:

$$S(R_i(bef), R_i(after)) = Agg_{j=1}^{p} S_{bef,after}(j) \tag{41}$$

where Agg denotes an aggregation operator and p is the number of inputs. A feasible choice is a t-norm [76], as a strong non-overlap along one single dimension is sufficient that the clusters do not overlap at all (hence are dissimilar, thus S should become a low value); this is because one dimension can be already sufficient for tearing the rules away in a geometric sense, see also [75].

4.2. Reducing the rule length dynamically with incremental feature weights

The concept of feature weighting, as described in Section 3.4, which has been used to define the weights w in the neurons construction according to (11), can also be used for advanced transparency and interpretability purposes. That is simply because features receiving a low weight close to 0 can be seen as unimportant for explaining the knowledge represented in the data and the associated class responses. Hence, they can be masked out when showing the rules according to their definition in (15) to an expert, which decreases the length of the rules and thus increases their understandability (see examinations in [8]). Furthermore, the features' impact can also be shown to experts as part of the rule definition in (15)– in an interactive environment, and experts may then better understand the conditions/relations for the classes. They may even manually change the impact of features (with an interpretable meaning for them) according to their experience before the model's adaptation continues: this will allow data and human knowledge integration in one evolving neuro-fuzzy system.

Fig. 4 shows an example of weight variation in a 7-dimensional data set. It is noted that there are dimensions of the problem extremely relevant to the problem during all interactions with new data, and there are high dynamics in the data, as can be seen, due to significant changes among a couple of samples. Feature 3 seems to be unimportant at the beginning of the stream but increases its importance after around 7–8 samples and then continues as the most important feature throughout the whole process. Four features, namely 1, 4, 6, and 7, become more and more unimportant over time, all receiving a weight smaller than 0.2; they can be finally ignored when showing the rules to an expert. That leaves three significant features towards the end of the process (stream), namely 2,3,5 with an importance level (weight) of $[0.4, 1, 0.25]$. Interestingly to see that Feature 2 is unimportant at the beginning, most important during the middle phase while getting mediocre important during the end of the process stream.

5. Test of pattern classification

All the tests were performed on a computer with the following settings: Core (TM) 2 Duo CPU, 2.27 GHz with 3-GB RAM.

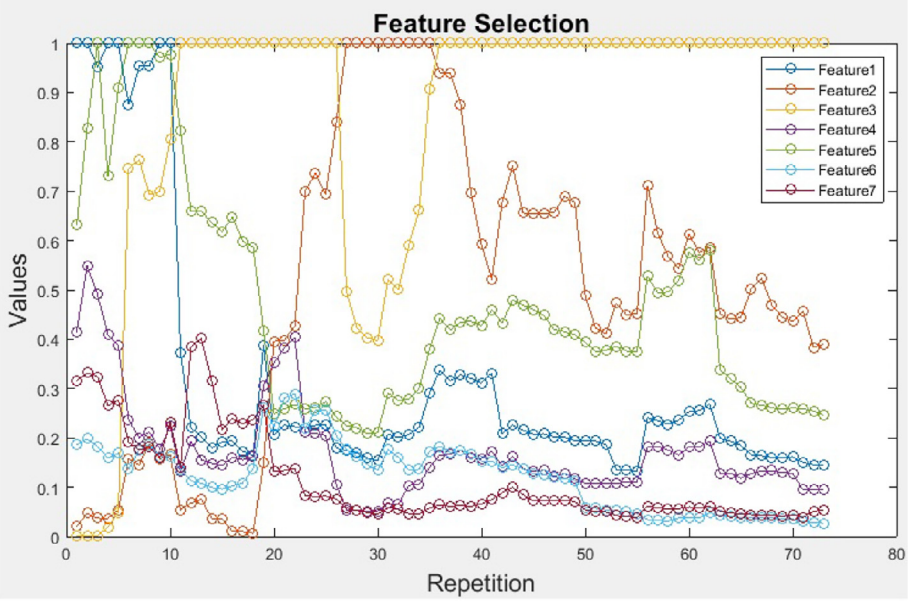


Fig. 4. Example of weight variation through incremental feature weighting (from a real-world process).

5.1. Assumptions and initial test configurations

To conduct the tests, real classification datasets were collected, stemming from different real-world applications, such as health, industry, and others. To evaluate the model's efficiency in different scenarios, data sets with different number of samples and input dimensionality will be used. It is also worth mentioning the use of a well-known data set so that the evolved model can be checked according to its interpretability capabilities.

The following tables show information concerning the experiments, introducing determinants such as the percentage of examples intended for the training and testing of fuzzy neural networks. The model proposed in this paper, called ENFS-Uni0, is empirically compared with related state-of-the-art methods as presented in Table 5, showing their parameters and default values as well as the references where they have been originally published. During an initial batch modeling stage, the optimal parameters were elicited through a 10-fold cross-validation procedure on all the parameter combinations established. This was carried out on the first 70% of the data, while the remaining 30% was used as separate test set. Thirty repetitions were performed for each data set to be validated by the models, and the result that maximized accuracy was selected for the model's final validation test. The parameters were varied in the following way:

- ALMMo-0: radius $r_o = [\sqrt{2-2cos(30^\circ)}, \sqrt{2-2cos(45^\circ)}, \sqrt{2-2cos(60^\circ)}, \sqrt{2-2cos(90^\circ)}]$.
- EFNHN: *Learning rate* $(\tau) = [0.01, 0.02, 0.03, 0.04, 0.05, 0.06]$; centers $m = [2, 4, 6, 8, 10]$.
- SPE-AHN: *Learning rate* $(\tau) = [0.01, 0.02, 0.03, 0.04, 0.05, 0.06]$; centers $m = [2, 4, 6, 8, 10]$.
- DDC-FNN: $r_1 = [1.0, 2.0, 3.0, 4.0, 5.0]$; bootstrap replications $b_t = [8, 16, 32, 64]$; consensus threshold $\lambda = [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]$.
- ENFS-Uni0 (our method): Bootstrap replications $b_t = [8, 16, 32, 64]$; consensus threshold $\lambda = [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]$.

The models were chosen through the nature of hybrid models published in high impact magazines and the responsiveness and interpretability of the models. The offline phase was performed with the training samples (70%), and the evolving phase was performed with the test samples (30%). In all models, the weights and bias were used randomly in the first and second layers in the interval between [0, 1], except for the model proposed in this paper, which uses the feature weighting concept described in Section 3.4.

A total of 30 experiments were performed with all models submitted to all test databases. In all tests and all models, the samples were shuffled in each test to demonstrate the models' actual capacity. Accuracy values (in percent) for the classification tests are presented in the results tables, accompanied by the standard deviation over the 30 repititions. The accuracy values are estimated by dividing the number of correctly classified samples through all samples.

Table 5
Hybrid models used in pattern classification tests.

Acron.	Context	Parameters + Defaults	Author
ALMMo-0	Ext. zero-order Autonomous Learning Multiple-Model neuro-fuzzy approach	$r_o = \sqrt{2-2cos(30^\circ)}$	Soares et al. [77]
EFNHN	Evolving fuzzy neural hydrocarbon networks	(τ) = 0.01, m = 6	Souza, Ponce and Lughofer [58]
SPE-AHN	Stochastic parallel extreme artificial hydrocarbon networks.	(τ) = 0.01, m = 4	Ponce et al. [78]
DDC-FNN	Fuzzy Neural Network based on Data Based Clustering and Nullneurons	r_1 = 3.0, bt = 16, λ = 0.6	de Campos Souza et al. [79]
ENFS-Uni0	Evolving Neuro-Fuzzy System proposed in this paper	$b_t = 16, \lambda$ = 0.6	De Souza, Lughofer

Table 6
Dataset used in the experiments.

Dataset	Init.	Feature	Train	Test	Classes
Haberman	HAB	3	214	92	2
Transfusion	TRA	4	523	225	2
Mammographic	MAM	5	581	249	2
Cryotherapy	CRI	6	63	27	2
Optical Interc. Network	OIN	9	448	192	2
German Credit	GER	14	390	168	2
Austism	AUT	19	204	88	2
Australian Credit	AUS	24	700	300	2
Ionosphere	ION	32	245	106	2
Iris	IRI	4	105	45	3
Wine	WIN	13	124	54	3
Glass	GLA	9	150	64	6
Ecoli	ECO	7	235	100	8
Credit Card	CCA	23	21,000	9,000	2

5.2. Dataset used in the tests

In Table 6 the real data sets extracted from the UCI Machine Learning repository[80] and Yeh and Lien [81] for classification problems are shown. These data sets have a good variation in terms of the number of input features (from small to larger problems), the number of classes (binary and multi-class) and the field of applications where they stem from.

5.3. Pattern classification tests

Table 7 shows the accuracy results for each data set (rows) among all methods (columns). The values highlighted in bold show the best individual test results for each row. It is remarkable that our method could outperform all others in nine out of 14 cases, whereas for the five remaining cases it was among the top performers and never significantly worse than the best related SoA methods. Thus, we can conclude that this paper's proposal meets the criteria for classifying standards, and this by acting in an evolving way to solve problems.

5.4. Examining interpretability of evolving rules

Immunotherapy treatment has been helping in several types of scientific approaches, mainly in treating warts and HPV (Human Papillomavirus). Therefore, acting to discover the best efficacy in patients is essential to avoid problems with the costs involved in the treatment and to be uncomfortable applying the approach, depending on the region where you want to destroy warts. To facilitate physicians and studies related to the viability of immunotherapeutic treatment, Khozeimeh et al. [82] developed a study in which it was possible to collect data from patients who underwent treatment and their respective final results on its assertiveness. The main features of the problem are sex (1-Man, 2- Woman), age, time (elapsed before treatment in the month), number of warts, type (1- Common,2- Plantar,3- Both), area (mm^2), induration diameter (initial test(mm)). The result of treatment (ROT) is the variable expected by the model (1-Yes, 2-No).

Table 8
Result of the classification of patterns with the immunotherapy dataset.

Offline Accuracy:	87.30 % (2.67)
Number of Original Fuzzy Neurons:	18.00
Number of Evolving Fuzzy Neurons:	7.00
Sensitivity:	40.00%
Specificity:	100.00%
AUC:	0.7000
Evolving Accuracy:	88.89% (2.23)

Their studies with that database obtained 80.70% accuracy when defining the final treatment status. Therefore, we used this as reference value for model accuracy to examine the assertiveness and interpretability of the evolving fuzzy neural network model. Simultaneously, the experiment has the same previous configurations, where 70 % of the samples were destined for the initial training of the model and the remainder for evolving the model on new samples.

The model trained on 30 repetitions obtained an average accuracy of 81.78%, with a standard deviation of 5.43%. Therefore, the approach brings results similar to the study that performed the data collection. During online training, a decrease in the number of neurons (rules) from 18 down to 11 could be observed due to

Table 7
Accuracy of the model in the tests performed.

Dataset	ENFS-Uni0	ALMMo-0	EFNHN	SPE-AHN	DDC-FNN
HAB	**74.45 (3.56)**	67.53 (4.82)	65.07 (11.03)	73.65 (2.93)	73.44 (4.33)
TRA	77.70 (3.12)	59.07 (2.50)	72.02 (12.58)	**79.16 (2.40)**	76.13 (2.67)
MAM	**83.95 (2.98)**	75.50 (3.13)	78.12 (4.81)	83.11 (2.09)	77.64 (3.33)
CRI	87.16 (5.56)	87.16 (5.21)	65.55 (11.05)	**88.14 (6.62)**	60.37 (11.59)
OIN	70.55 (4.65)	68.10 (3.17)	62.30 (4.44)	**74.78 (3.31)**	59.80 (3.73)
GER	**84.49 (7.08)**	77.74 (2.91)	65.60 (8.20)	82.81 (2.95)	58.96 (3.04)
AUT	**95.90 (1.94)**	86.21 (3.56)	83.14 (10.99)	93.52 (2.67)	78.82 (6.91)
AUS	**75.20 (2.48)**	66.26 (2.28)	69.74 (3.05)	74.61 (1.64)	69.43 (2.40)
ION	**89.71 (2.93)**	89.39 (3.00)	89.61 (2.61)	81.17 (3.69)	67.84 (6.82)
IRI	**96.44 (2.63)**	94.22 (2.57)	86.07 (13.15)	94.96 (6.19)	86.17 (6.02)
WIN	84.40 (9.92)	**95.22 (2.20)**	58.05 (13.71)	90.56 (4.80)	81.02 (4.12)
GLA	**87.92 (2.11)**	83.95 (4.54)	32.08 (10.45)	73.07 (5.50)	84.47 (2.56)
ECO	89.18 (2.77)	80.82 (2.83)	30.52 (10.28)	49.37 (4.50)	**92.33 (1.36)**
CCA	**81.55 (3.15)**	72.87 (0.36)	77.74 (1.74)	81.40 (0.22)	78.07 (0.48)

merging concepts in the ADPA. Table 8 presents the results obtained in one of the 30 experiments.

Fig. 5 demonstrates the evolution of accuracy for the evolving phase performed on 30 repetitions, which is compared with the accuracy achieved by the off-line model. In the Fig. 5, it is possible to identify that the final results vary significantly in some repetitions, but in most of them, they are stable and allow a more cohesive assessment on the nature of the model's performance. It can be noted that the comparison of the results between the pure off-line and the evolving model shows a similar behavior, whereas some unexpected low accuracies could be more often observed for the off-line model. Also the standard deviation among all repetitions were lower for the evolving model than for the off-line trained one.

The assessment of feature weights (and their change over time) can also help to understand the immunotherapy problem's dynamics. Fig. 6 shows the comparison of the feature weights achieved during the offline training phase and at the end of the evolving phase. It can be seen that the feature 'in months before starting the treatment' (simply termed as 'Time' in the figure) is the most important feature carrying most of the information for the classification problem (and this without variation during the evolving phase → stable important feature). However, it should be noted that after the evolving phase, the relevance of the patient's gender, the number of warts, and the induration diameter contribute more significantly to the model (and thus to the input-class relation of the classification problem) than before because their weights increased significantly (for the initial model, only 'Age' and 'Time' significantly contributed, with little impact of 'Type'). Therefore, it is possible to conclude that the ENFS-Uni0 model proposed in this paper may helps to interpret which dimensions must collaborate for the results of immunotherapy treatment.

As an example, we show the initial Rule 1 after the off-line training of the model:

Rule 1. If (Sex is Man) with impact 0.09 OR (Age is Middle age) with impact 0.299 OR (Time is Elevated) with impact 1.000 OR (Number Warts is Medium) with impact 0.143 OR (Type is Both) with impact 0.063 OR (Area is Large) with impact 0.078 OR (Induration is Large) with impact 0.315 THEN (Result of Treatment is Yes (with certainty: 5.667)).

The impacts of the features are the same for all rules, as we used a global criterion according to the description in Section 3.4. From Rule 1 (and also from Fig. 6) we can realize that only 4 of the 7 features are significantly active (with weights above 0.1), namely

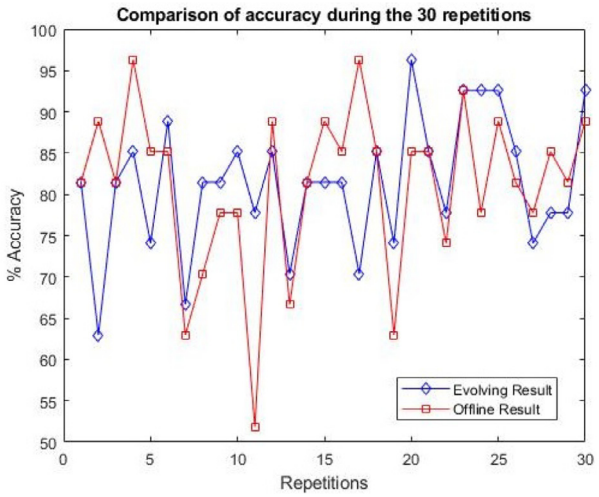


Fig. 5. Accuracy results obtained for 30 repetitions (shuffles) on the training data.

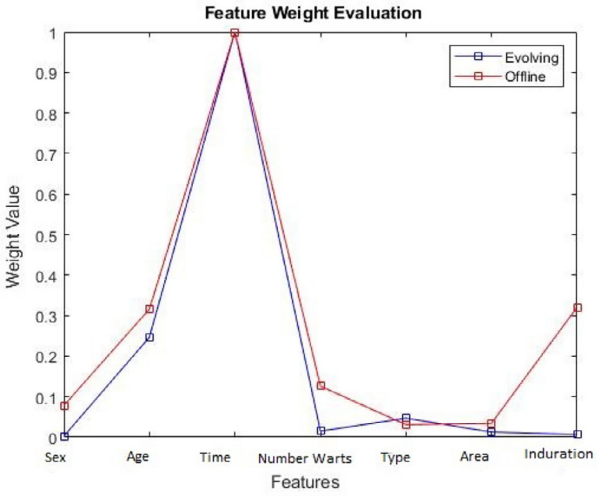


Fig. 6. Feature weights (importance levels) for off-line and further evolving case.

'Age', 'Time', 'Number Warts' and 'Induration'. Hence, we can reduce the rule length by discarding these from the antecedents, which leads to the following (more transparent) description of the seven rules:

Rule 1. If (Age is Middle age) with impact 0.312 OR (Time is Moderate) with impact 1.000 OR (Number Warts is Medium) with impact 0.123 OR (Induration is High) with impact 0.315 THEN Result of Treatment is Yes (with certainty: 1)).

Rule 2. If (Age is Advanced) with impact 0.312 OR (Time is Elevated) with impact 1.000 OR (Number Warts is High) with impact 0.123 OR (Induration is High) with impact 0.315 THEN (Result of Treatment is NO (with certainty: −0.4)).

Rule 3. If (Age is Advanced) with impact 0.312 AND (Time is Recent) with impact 1.000 AND (Number Warts is High) with impact 0.123 AND (Induration is High) with impact 0.315 THEN (Result of Treatment is YES (with certainty: 0.9)).

Rule 4. If (Age is Advanced) with impact 0.312 AND (Time is Elevated) with impact 1.000 AND (Number Warts is High) with impact 0.123 AND (Induration is High) with impact 0.315 THEN (Result of Treatment is No (with certainty: −1)).

Rule 5. If (Age is Advanced) with impact 0.312 AND (Time is Moderate) with impact 1.000 AND (Number Warts is High) with impact 0.123 AND (Induration is High) with impact 0.315 THEN (Result of Treatment is Yes (with certainty: 0.8)).

Rule 6. If (Age is Advanced) with impact 0.312 OR (Time is Elevated) with impact 1.000 OR (Number Warts is High) with impact 0.123 OR (Induration is High) with impact 0.315 THEN (Result of Treatment is NO (with certainty: −0.8)).

Rule 7. If (Age is Youthful) with impact 0.312 AND (Time is Elevated) with impact 1.000 AND (Number Warts is High) with impact 0.123 AND (Induration is High) with impact 0.315 then (Result of Treatment is YES (with certainty: 0.8)).

It is important to emphasize that the consequent level of certainty is correlated to the possible expected exit from the problem. For 'YES' answers, the closer to 1, the greater the certainty about the rule. In the case of 'NO', the closer to −1, the greater the certainty about the result presented in the rule.

After training in the evolution of the model, the rules were duly adapted together with the impacts of the features on the classification response (see Fig. 6). It is remarkable that the impact of two additional features were decreased towards 0 (thus, they became negligible), such that it finally turned out that only 'Age' and 'Time' mattered for explaining the classification problem. This discovery follows the same reasoning as what was said in Khozeimeh et al.,

Table 9
Interpretability with respect to (degree of) changes in fuzzy neurons during the evolution phase.

Rule 1 did change in 6 membership functions and by a degree of 0.0687 with consequent change from Yes to No.
Rule 2 did change in 7 membership functions and by a **degree of 0.4884** with consequent change from No to Yes.
Rule 3 did change in 6 membership functions and by a degree of 0.0214 with no consequent change.
Rule 4 did change in 7 membership functions and by a degree of 0.0523 with no consequent change.
Rule 5 did change in 7 membership functions and by a degree of 0.1107 with no consequent change.
Rule 6 did change in 6 membership functions and by a degree of 0.0495 with consequent change from No to Yes.
Rule 7 did change in 7 membership functions and by a degree of 0.0543 with consequent change from Yes to No.

who defined using the technique information gain that the three features most relevant to the immunotherapy problem were time, induration, and age. It is worth mentioning that Khozeimeh's study was based on a group of experts to define the initial dimensions of the problem. Hence, the seven further evolved rules on new data can be compactly (with two antecedent parts only) written as:

Rule 1. If (Age is Advanced) with impact 0.246 OR (Time is Elevated) with impact 1.000 THEN (Result of Treatment is NO (with certainty 0.1)).

Rule 2. If (Age is Advanced) with impact 0.246 OR (Time is Moderate) with impact 1.000 THEN (Result of Treatment is Yes (with certainty 0.9)).

Rule 3. If (Age is Advanced) with impact 0.246 AND (Time is Very-Elevated) with impact 1.000 THEN (Result of Treatment is Yes (with certainty 0.6)).

Rule 4. If (Age is Very-Advanced) with impact 0.246 AND (Time is Very-Elevated) with impact 1.000 THEN (Result of Treatment is No (with certainty −1)).

Rule 5. If (Age is Middle age) with impact 0.246 AND (Time is Moderate) with impact 1.000 THEN (Result of Treatment is No (with certainty −0.3)).

Rule 6. If (Age is Very-Advanced) with impact 0.246 AND (Time is Elevated) with impact 1.000 THEN (Result of Treatment is Yes (with certainty 1)).

Rule 7. If (Age is Middle age) with impact 0.246 AND (Time is Very-Elevated) with impact 1.000 THEN (Result of Treatment is No (with certainty −0.8)).

Evolving training altered neuron centers and sigmas and changed the relevance value of each precedent of the rule when receiving new data on the problem. Therefore, the model can be seen with similar human learning behavior that updates ideas and concepts after learning and understanding new problem data. Finally, the similarity of the neurons before and after evolution phase (a high value indicates a low degree of change) are demonstrated in the Table 9, and also how many membership functions (out of 7 in sum) are changed. As can be seen, except in the case of Rule 2, the antecedents changed only marginally, as similarity is reported to be close to 1 in all other cases. This means that the input space partitioning through local clouds hardly changed its nature during new samples, while the output weights (certainty in the classes) did (as discussed above). From this it can be concluded that this (small) data set shows a target concept drift and that it would be more advantageous to collect more samples in order to gain greater confidence in the knowledge extracted so far.

The behavior of the model when the new samples are used samples for evaluation and model updating afterwards (leading to ahead-predictive accumulated accuracy trend lines) can be seen in the Fig. 7. Thus, this figure shows how the off-line and the on-line models are expected to perform in a predictive step-ahead procedure. It is easy to realize that the off-line (and not further evolved) model gets outperformed from around Sample 15 on (because its accuracy trend line stays way below of that one achieved by the incrementally updated online model). Thus, we can conclude that the model update and evolution is important to keep accuracy on a higher level, an issue which becomes even more severe on the data sets evaluated in the subsequent section.

5.5. Further results on online model updating and evolution

In order to check the on-line behavior of the evolving model on a sample-wise basis, we performed tests where the accuracy on new on-line samples is permanently updated and the model evolved on the new samples after making a prediction on them. Concretely speaking, we observed the accumulated accuracy trend lines in an interleaved-test-and-then-train scenario, which is a well-known and widely used performance measures in the data stream modeling community, see [44]. These trend lines were compared among various related evolving approaches and also with an initial off-line model, which was not further evolved a new data samples, but kept fully static for the complete bunch of new (on-line) samples. According to the trend lines, we can realize any convergence behavior of a model (higher accuracy trend lines over time) or any unexpected down-trends etc.

5.5.1. SQL cyber-attacks

For binary classification tests, a datas et with 24917 samples of SQL Injection cyber-attacks was used. This type of cyber-threat is generated through SQL code commands intending to destroy a system with a database or obtaining confidential data from a system through the use of SQL codes and commands. In this experiment, the data were obtained with python-sqlparse[1] and a re-sampling technique called SMOTE [83], generating new samples to make the data set more balanced for evaluations of the models [84]. The main dimensions of the data set for SQL Injection attacks are Length, Entropy, Malicious, Legit, and Difference Score. Consequently, the data set deals with the classification of binary problems where the label 0 means that there was no attack, and class 1 indicates a SQL Injection attack.

For the test execution, the model proposed in this paper was used in the offline and online versions. Additionally, we used one time the standard technique of random definition of weights in the interval between 0 and 1 and one time the incremental feature weighting concept described in Section 3.4. That will make it possible to assess the stability generated by defining neuron weights in the first layer of the model based on assessments of the impact of the features. In conjunction with these versions of the ENFS-Uni0 modelling approach proposed in the paper, the evolving clustering technique called Self-Organized Direction Aware Data Partitioning Algorithm (SODA) as proposed by Gu et al. [85], and the ALMMo-0 approach (extended Zero-order Autonomous Learning Multiple-Model neuro-fuzzy approach) as proposed in [77]) were used as benchmark algorithms.

Fig. 8 presents the final results of the model, sample by sample, where 500 initial samples were destined for training of the initial (off-line) model and the rest for permanent prediction and model updated, according to the interleaved-test-and-then-train scenario.

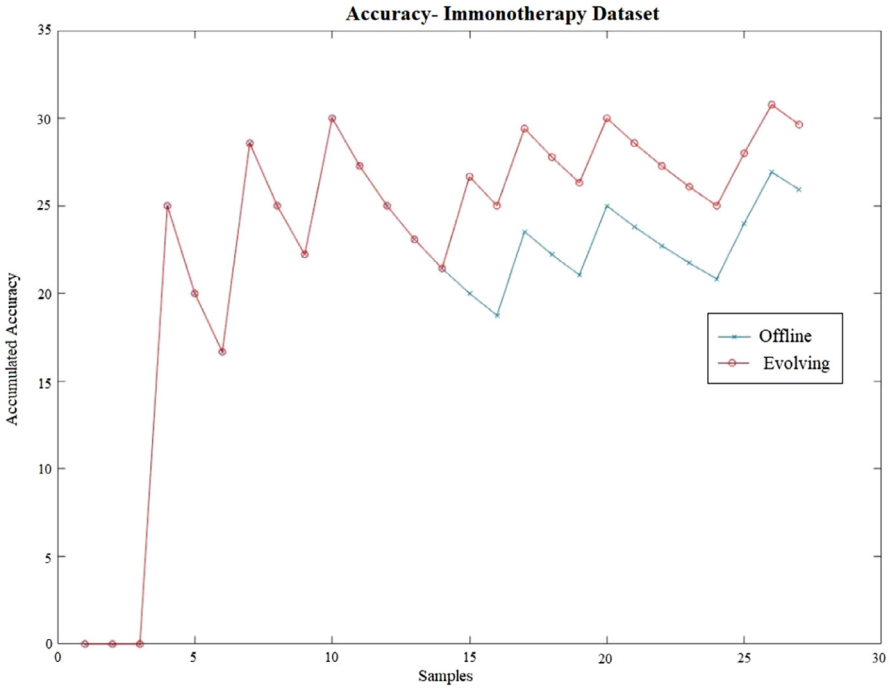


Fig. 7. Accuracy Trend Lines for the Immunotherapy data set.

The Fig. 8 highlights the efficiency of online models compared to offline models, where models with an evolving approach obtained the best results of accumulated accuracy. This test highlights the efficiency of defining weights of neurons in the first layer based on the problem's features when comparing random weight and feature weight approaches. Furthermore, it is possible to understand the relevance of the problem's feature when that sees the first layer neurons become fundamental for the problem's adequate resolution.

The first factor to be highlighted in Fig. 8 is the difference between the offline model's behavior and that of the evolving model. As expected, the offline model begins its assessment with excellent results, but only for a few new on-line samples. Subsequently, its ability to perform reasonable predictions deteriorates soon. This behavior was observed in both offline versions used in this experiment, so with the inclusion of feature importance levels calculated through (32) and without it by just assigning random weights. The former can significantly improve the prediction behavior of the model, as the accuracy deterioration is not so drastic as it is in the case of random weights. But, its trend line is still much worse than all of the evolving versions and methods, which can show an increasing accuracy trend line behavior (after the initial phase) up to a saturation level around 95–97%.

The SODA approach could come close to the performance of our method, but its accuracy trend line finally always stayed slightly below those ones achieved by our methods—towards the end of the stream, the scissor more opened up leading to a trend line about 2% worse than our method. Finally, the behavior of the ALMMo-0 model worsens considerably starting around Sample 12000. A reason for this effect is perhaps an incorrect update of the model parameters.

5.5.2. Iris flower classification

A second test to be carried out in this paper is the entry of new information and, consequently, the aspect of new classes' discovery over time. For this, the data set that deals with three different types of iris flowers was used. In this experiment, an initial offline model was trained using only one iris flower class (first 30 samples). The results obtained can be seen in Fig. 9.

The offline model classified the samples of the first flower very well, but could not identify the second and third iris flowers, thus showing a permanently decreasing accuracy trend line. On the other hand, the online model proposed in this paper obtained excellent results for accuracy, as it was able to learn the nature of new samples and integrate the two other existing classes on demand and on the fly once samples of them were arising in the stream and thus sent into the update engine. A very small downtrend could be observed when a new class starts, but which is quickly increasing again once the new class is established in the model with further samples. Therefore, we can say that the evolving model acts dynamically in determining new classes of the problem according to its complex nature. This factor represents advances in the identification of new trends in data sets and, at the same time, improves the interpretive capacity of the model, which can extract new knowledge from new samples as the nature of the problem expands. In fact, two new fuzzy-neurons were autonomously evolved for the two newly arising classes (Class 2 and 3).

5.5.3. Human fall data set

Finally, to further verify the model's ability to identify classes that were not provided in the initial training phase, an experiment was conducted with a database generated in the paper by de Campos Souza et al. [86], originated from the study of falls monitored by markers carried out by Martínez-Villaseñor et al. [87]. The data set initially had 45 dimensions, but was reduced to 15 features by pre-selecting most important features. This database has 258,114 records of falls monitored by cameras and devices attached to the subjects who participated in the test by simulating different falls. As a result, 11 types of falls were raised and, additionally, a feature to represent an unexpected behavior. So, this data set is an excellent example for evolving a model in the case when new classes appear dynamically on the fly during a stream mining process (so, when not all classes are available during the initial modeling stage), and to check how the model performs in such cases.

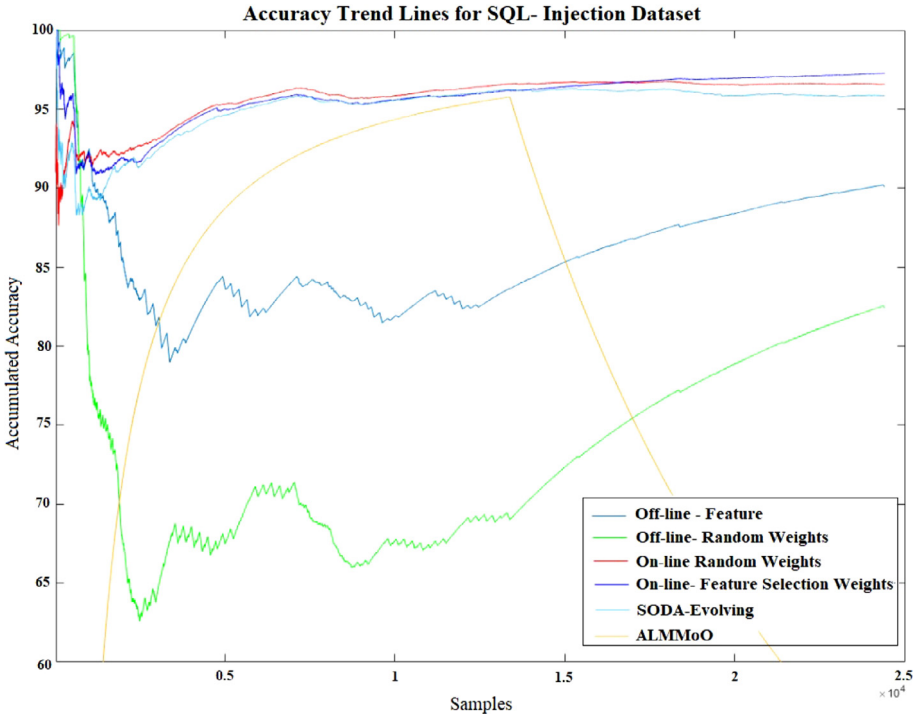


Fig. 8. Accuracy trend lines for SQL Injection cyber-attacks data stream.

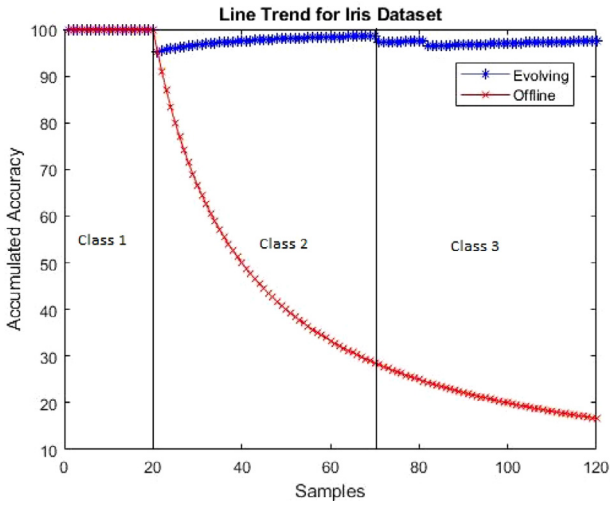


Fig. 9. Accuracy Trend Lines for the Iris data set; comparison of evolving model and off-line trained model.

The experiments' execution can be seen in Fig. 10, where the difference between evolving training and gffline training is highlighted. The model that worked with offline training could not identify new classes, and thus always showed a significantly decreasing accuracy behavior when a new class was introduced (sawtooth-like trend line), apart from the fact that the accuracy went down quickly for the first new on-line samples according to dynamics in the process. These downtrends did not happen when using the evolving model, which showed a very stable behavior close to 100% during the whole stream.

The evolving model addressed the complex problem of multiple outputs efficiently, learning in a coherent way to identify the model's outputs. However, the offline model did not achieve the same success, most likely because it only had three outputs in the training phase, making the model with low generalization capacity to discover new classes. The behavior shown by the lines in Fig. 10 can be justified by the confusion matrices that were generated from the experiments (Fig. 11 and Fig. 12). Note that only class 11 was identified by the model in the offline model (as this was the only class present in the first 500 samples used for initial training). This indicates that this version of the model is not efficient in classifying samples from new classes. However, as can be realized from the confusion matrix of the evolving model, the fuzzy neural network correctly identified new classes, new behaviors, evidencing the importance of using adaptive training to improve the model's performance, as the highest entries are arranged in the main diagonal of the confusion matrix (= correctly classified samples to each class).

6. Conclusion

This paper presented an evolving fuzzy neural network capable of solving complex pattern classification problems with a high degree of assertiveness and interpretability. For this, the model uses the construction of fuzzy neurons through an adaptive fuzzification process and an evolving training procedure, which is able to determine an adequate number of neurons autonomously. The construction process allows to combine the antecedent parts of the rules with AND- and OR-statements, which opens up new possibilities for model interpretability compared to classical evolving neuro-fuzzy approaches allowing only AND-connections. The procedure for obtaining weights in the first layer through procedures related to incremental feature weighting gave an interpretive view of feature relevance in the neurons/rules. Furthermore, this is a deterministic approach, abandoning the usage of random weights, which would make the model hardly interpretable in a consistent way (as inducing different outcomes for different training runs). Finally, the neural aggregation network training through an indicator-based recursive least squares in order to achieve an

incremental, adaptive one-versus-rest classification model allows the best adaptation of the output weights of the neurons according to the change in the information contained in the data stream.

The first experiment evaluated the model's classification capacity with data sets commonly applied in the pattern classification literature. Of the 14 datasets (with complexities, features, and number of different samples), the proposed model was the most efficient in 9 of them, proving the model's ability to act in this type of problem. In the data set immunotherapy experiment, the ability to improve the model's accuracy in the comparison between offline training and evolution training was proven. This test also showed the ability to optimize the fuzzy rules' background, building them

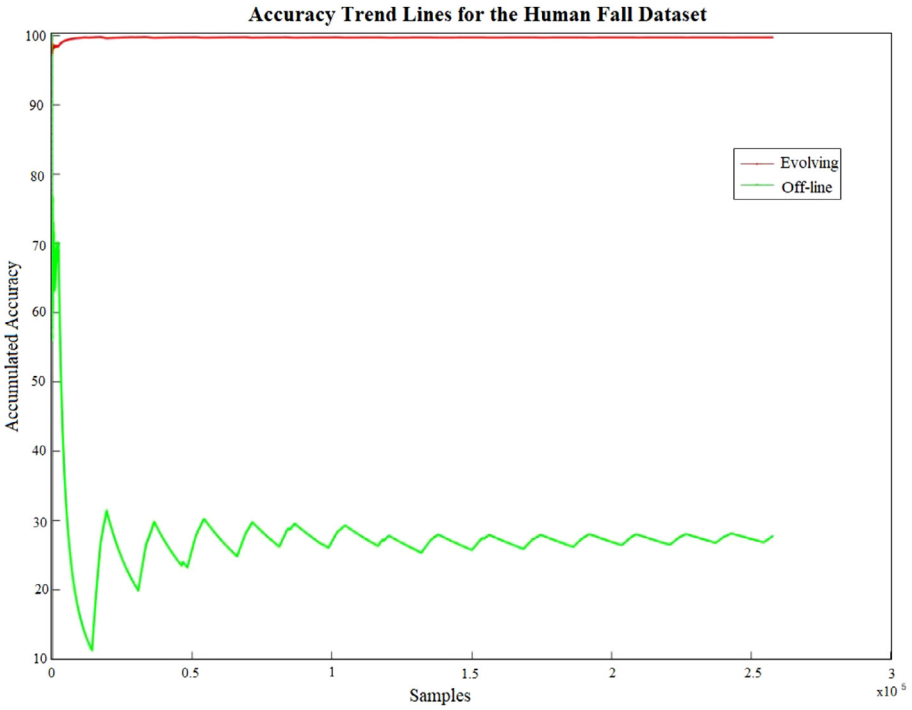


Fig. 10. Accuracy Trend Lines for the Human Fall data set.

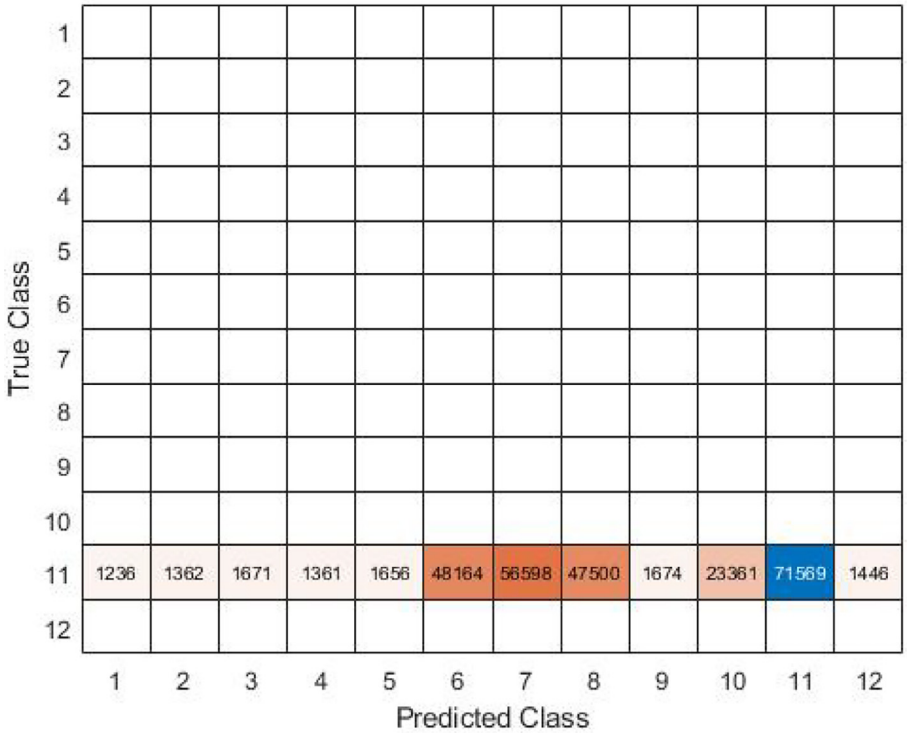


Fig. 11. Confusion Matrix for Human Fall data set when using off-line model for classifying new online samples.

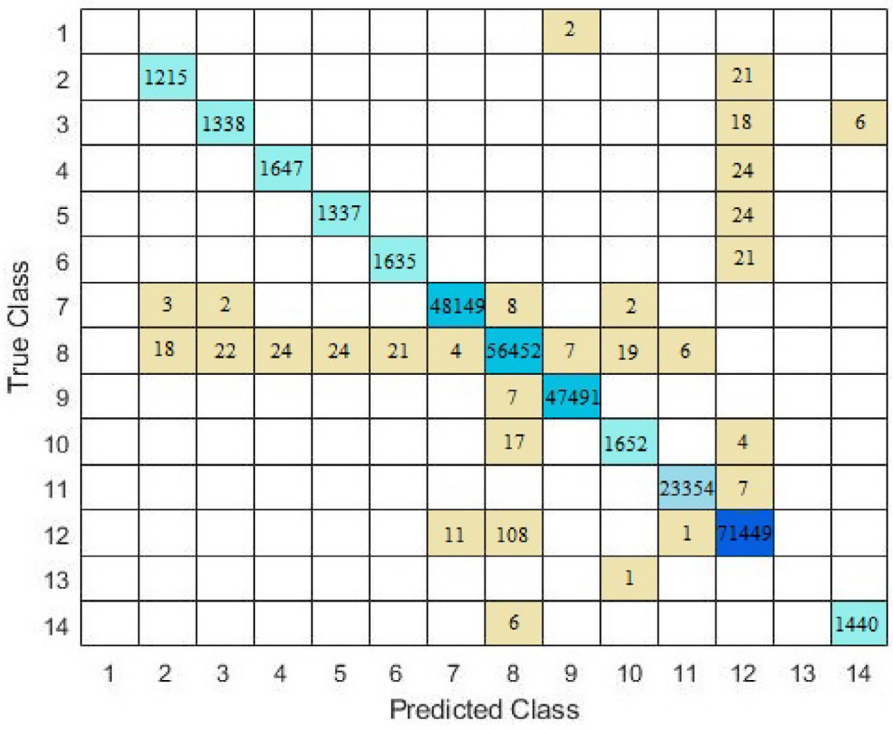


Fig. 12. Confusion Matrix for Human Fall data set when using evolving model for classifying new samples.

only with the relevant dimensions of the problem, as turning out as one key feature of our approach due to the incremental feature weighting concept: from the listed rules in Section 5.4, it can be clearly seen that only 4 and, after evolution, only 2 out of the 7 original input features are relevant to describe the classes. The knowledge extracted from the data set (and the changing dynamics behind) should be thus well interpretable by humans. In the third test, the stream data approach was successfully validated, where the best results (blue line) were from the model proposed in this paper, showing the accumulated predictive ahead accuracy trends lines over time. The comparison with related evolving neuro-fuzzy models in the literature demonstrates the model's assertive capacity to evolve the model parameters as new data are submitted to the problem, and also the much better performance than a classical batch offline model. The fourth test (iris dataset) obtained results relevant to the literature because the model in its initial approach was presented with only data from one class and our evolving model was able to identify the two other remaining classes as keeping the accuracy on a high level (which was not the case for a classical off-line trained model showing decreasing accuracy over the samples, see Fig. 9). That proves that the model proposed in the paper can discover new classes in problems to be evaluated—it also automatically generated two new neurons for the two arising classes. Finally, the fifth test also proves the model's ability to identify new classes in a problem wherein the initial training data from three classes were submitted, and the model, in its evolving version, was able to identify the other 14 classes arising latter in the stream (so, most of the classes are not known in advance and thus not being 'trainable' into a classical off-line modelling approach).

A limitation of our approach currently is that, although we apply an advanced construction of neurons through uni-norms and thus allow OR-connections of rule antecedent parts, a single rule can embed only either AND or only OR connections, but not a mixture of them, e.g., in form of a conjunctive normal form, which may open up additional interpretability capabilities. Another limitation could be that no pruning/merging of clouds (neurons) is conducted during on-line mode, which could make the model unnecessary complex whenever older states (reflected in older learned neurons) become outdated.

Future works will address the following issues: (i) overcoming the limitations discussed above, (ii) performing a localization of the feature importance levels (weights for neurons construction) per rule (currently, the same impact is used across all rules), and (iii) dealing with the integration of expert's input on sample and structural level to improve accuracy trend lines in specific situations (such as experts may have different certainty in class labels, may have some a priori knowledge about a new arising class in the stream, may disagree with some (aspects in) rules shown to her/him etc.).

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

CRediT authorship contribution statement

Paulo Vitor de Campos Souza: Conceptualization, Methodology, Software, Writing - original draft, Visualization, Resources, Writing - review & editing. **Edwin Lughofer:** Data curation, Writing - original draft, Visualization, Investigation, Formal analysis, Supervision, Validation, Writing - review & editing, Project administration, Funding acquisition.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors acknowledge the Austrian Science Fund (FWF): contract number P32272-N38, acronym IL-EFS.

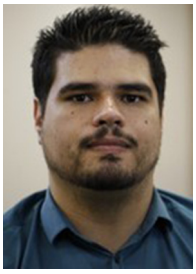
References

[1] L. Harasim, Learning theory and online technologies, Taylor & Francis, 2017.
[2] M.I. Jordan, T.M. Mitchell, Machine learning: trends, perspectives, and prospects, Science 349 (2015) 255–260.
[3] M. Sayed-Mouchaweh, E. Lughofer, Learning in Non-Stationary Environments: Methods and Applications, Springer, New York, 2012.
[4] W. Pedrycz, F. Gomide, Fuzzy Systems Engineering: Toward Human-Centric Computing, John Wiley & Sons, Hoboken, New Jersey, 2007.
[5] P.V. de Campos Souza, Fuzzy neural networks and neuro-fuzzy networks: a review the main techniques and applications used in the literature, Appl. Soft Comput. 92 (2020) 106275.
[6] N. Kasabov, Evolving fuzzy neural networks for supervised/unsupervised online knowledge-based learning, IEEE Trans. Syst. Man Cybern. Part B (Cybernetics) 31 (2001) 902–918.
[7] P. Angelov, D. Filev, An approach to online identification of Takagi-Sugeno fuzzy models, IEEE Trans. Syst. Man Cybern. Part B (Cybernetics) 34 (2004) 484–498.
[8] E. Lughofer, On-line assurance of interpretability criteria in evolving fuzzy systems–achievements, new concepts and open issues, Inf. Sci. 251 (2013) 22–46.
[9] P.P. Angelov, X. Zhou, Evolving fuzzy-rule-based classifiers from data streams, IEEE Trans. Fuzzy Syst. 16 (2008) 1462–1475.
[10] I. Škrjanc, J.A. Iglesias, A. Sanchis, D. Leite, E. Lughofer, F. Gomide, Evolving fuzzy and neuro-fuzzy approaches in clustering, regression, identification, and classification: a survey, Inf. Sci. 490 (2019) 344–368.
[11] E. Lughofer, Evolving fuzzy systems—fundamentals, reliability, interpretability and useability, in: P. Angelov (Ed.), Handbook of Computational Intelligence, World Scientific, New York, 2016, pp. 67–135.
[12] Y.-Y. Lin, J.-Y. Chang, C.-T. Lin, Identification and prediction of dynamic systems using an interactively recurrent self-evolving fuzzy neural network, IEEE Trans. Neural Networks Learn. Syst. 24 (2012) 310–321.
[13] P.-C. Chang, C.-Y. Fan, J.-J. Lin, Monthly electricity demand forecasting based on a weighted evolving fuzzy neural network approach, Int. J. Electr. Power Energy Syst. 33 (2011) 17–27.
[14] Y.-T. Liu, Y.-Y. Lin, S.-L. Wu, C.-H. Chuang, C.-T. Lin, Brain dynamics in predicting driving fatigue using a recurrent self-evolving fuzzy neural network, IEEE Trans. Neural Networks Learn. Syst. 27 (2015) 347–360.
[15] J. Tang, Y. Zou, J. Ash, S. Zhang, F. Liu, Y. Wang, Travel time estimation using freeway point detector data based on evolving fuzzy neural inference system, PloS one 11 (2016) e0147263.
[16] L. Decker, D. Leite, L. Giommi, D. Bonacorsi, Real-time anomaly detection in data centers for log-based predictive maintenance using an evolving fuzzy-rule-based approach, arXiv preprint arXiv:2004.13527 (2020)..
[17] M.M. Ferdaus, M. Pratama, S.G. Anavatti, M.A. Garratt, E. Lughofer, Pac: a novel self-adaptive neuro-fuzzy controller for micro aerial vehicles, Inf. Sci. 512 (2020) 481–505.
[18] D. Leite, R. Palhares, C.S. Campos, F. Gomide, Evolving granular fuzzy model-based control of nonlinear dynamic systems, IEEE Trans. Fuzzy Syst. 23 (2015) 923–938.
[19] G. Andonovski, P. Angelov, S. Blazic, I. Skrjanc, A practical implementation of robust evolving cloud-based controller with normalized data space for heat-exchanger plant, Appl. Soft Comput. 48 (2016) 29–38.
[20] P.V. de Campos Souza, E.A. Soares, A.J. Guimarães, V.S. Araujo, V.J.S. Araujo, T.S. Rezende, Autonomous data density pruning fuzzy neural network for optical interconnection network, Evolving Syst. (2020) 1–13.
[21] P.V. de Campos Souza, A.J. Guimaraes, V.S. Araujo, T.S. Rezende, V.J.S. Araujo, Incremental regularized data density-based clustering neural networks to aid in the construction of effort forecasting systems in software development, Appl. Intell. 49 (2019) 3221–3234.
[22] P. Angelov, X. Gu, Deep rule-based classifier with human-level performance and characteristics, Inf. Sci. 463–464 (2018) 196–213.
[23] J. Rubio, A. Bouchachia, MSAFIS: an evolving fuzzy inference system, Soft. Comput. 21 (2015) 2357–2366.
[24] D. Ge, X.-J. Zeng, Learning data streams online-an evolving fuzzy system approach with self-learning/adaptive thresholds, Inf. Sci. 507 (2020) 172–184.
[25] D. Leite, R. Palhares, C.S. Campos, F. Gomide, Optimal rule-based granular systems from data streams, IEEE Trans. Fuzzy Syst. 28 (2020) 583–596.
[26] R. Precup, T. Teban, A. Albu, A. Borlea, I.A. Zamfirache, E.M. Petriu, Evolving fuzzy models for prosthetic hand myoelectric-based control, IEEE Trans. Instrum. Meas. 69 (2020) 4625–4636.
[27] D. Ge, X.-J. Zeng, Learning data streams online – an evolving fuzzy system approach with self-learning/adaptive thresholds, Inf. Sci. 507 (2020) 172–184.
[28] D. Ge, X. Zeng, A self-evolving fuzzy system which learns dynamic threshold parameter by itself, IEEE Trans. Fuzzy Syst. 27 (2019) 1625–1637.
[29] M. Ashrafi, D.K. Prasad, C. Quek, IT2-GSETSK: an evolving interval Type-II TSK fuzzy neural system for online modeling of noisy data, Neurocomputing 407 (2020) 1–11.
[30] M.O. Camargos, I. Bessa, M.F.S.V. D'Angelo, L.B. Cosme, R.M. Palhares, Data-driven prognostics of rolling element bearings using a novel error based evolving takagi-sugeno fuzzy model, Appl. Soft Comput. 96 (2020) 106628.
[31] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, K. Ghedira, Discussion and review on evolving data streams and concept drift adapting, Evolving Syst. 9 (2017) 1–23.
[32] F. Hamker, RBF learning in a non-stationary environment: the stability-plasticity dilemma, in: R. Howlett, L. Jain (Eds.), Radial basis function networks 1: recent developments in theory and applications, Physica Verlag, Heidelberg, New York, 2001, pp. 219–251.
[33] E. Lughofer, M. Sayed-Mouchaweh, Autonomous data stream clustering implementing incremental split-and-merge techniques—towards a plug-and-play approach, Inf. Sci. 204 (2015) 54–79.
[34] D. Dovzan, V. Logar, I. Skrjanc, Implementation of an evolving fuzzy model (eFuMo) in a monitoring system for a waste-water treatment process, IEEE Trans. Fuzzy Syst. 23 (2015) 1761–1776.
[35] A.M. Silva, W. Caminhas, A. Lemos, F. Gomide, A fast learning algorithm for evolving neo-fuzzy neuron, Appl. Soft Comput. 14 (2014) 194–209.
[36] A. Lemos, W. Caminhas, F. Gomide, Multivariable gaussian evolving fuzzy modeling system, IEEE Trans. Fuzzy Syst. 19 (2011) 91–104.
[37] E. Lughofer, C. Cernuda, S. Kindermann, M. Pratama, Generalized smart evolving fuzzy systems, Evolving Syst. 6 (2015) 269–292.
[38] P. Akella, Structure of n-uninorms, Fuzzy Sets Syst. 158 (2007) 1631–1651.
[39] R.R. Yager, A. Rybalov, Uninorm aggregation operators, Fuzzy Sets Syst. 80 (1996) 111–120.
[40] T. Calvo, B. De Baets, J. Fodor, The functional equations of frank and alsina for uninorms and nullnorms, Fuzzy Sets Syst. 120 (2001) 385–394.
[41] E.S. Palmeira, B. Bedregal, Extension of fuzzy logic operators defined on bounded lattices via retractions, Comput. Math. Appl. 63 (2012) 1026–1038.
[42] X. Gu, P.P. Angelov, J.C. Príncipe, A method for autonomous data partitioning, Inf. Sci. 460 (2018) 65–82.
[43] R. Rosa, F. Gomide, D. Dovzan, I. Skrjanc, Evolving neural network with extreme learning for system modeling, IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS) 2014 (2014) 1–7.
[44] A. Bifet, G. Holmes, R. Kirkby, B. Pfahringer, MOA: massive online analysis, J. Mach. Learn. Res. 11 (2010) 1601–1604.
[45] A. Lemos, W. Caminhas, F. Gomide, A fast learning algorithm for uninorm-based fuzzy neural networks, in: Fuzzy Information Processing Society (NAFIPS), 2012 Annual Meeting of the North American, IEEE, 2012, pp. 1–6..
[46] E. Klement, R. Mesiar, E. Pap, Triangular Norms, Kluwer Academic Publishers, Dordrecht Norwell New York London, 2000.
[47] H. Zhou, X. Liu, Characterizations of (u2, n)-implications generated by 2-uninorms and fuzzy negations from the point of view of material implication, Fuzzy Sets Syst. 378 (2020) 79–102.
[48] J.C. Fodor, R.R. Yager, A. Rybalov, Structure of uninorms, Int. J. Uncertainty Fuzziness Knowl.-Based Syst. 5 (1997) 411–427.
[49] S.-K. Hu, Z.-F. Li, The structure of continuous uni-norms, Fuzzy Sets Syst. 124 (2001) 43–52.
[50] Y.-M. Wang, H.-W. Liu, On the distributivity equation for uni-nullnorms, Kybernetika 55 (2019) 24–43.
[51] P. Drygaś, E. Rak, Distributivity equation in the class of 2-uninorms, Fuzzy Sets Syst. 291 (2016) 82–97.
[52] E. Rak, The modularity equation in the class of 2-uninorms, in: Intelligent Systems' 2014, Springer, 2015, pp. 45–54..
[53] W. Zong, Y. Su, H.-W. Liu, B. De Baets, On the structure of 2-uninorms, Inf. Sci. 467 (2018) 506–527.
[54] Y.-M. Wang, F. Qin, Distributivity for 2-uninorms over semi-uninorms, Int. J. Uncertainty Fuzziness Knowl.-Based Syst. 25 (2017) 317–345.
[55] F.-X. Zhang, E. Rak, J. Bazan, On the distributivity of continuous triangular norms and triangular conorms with respect to 2-uninorms, Fuzzy Sets Syst. 395 (2019) 168–177.
[56] W. Pedrycz, Neurocomputations in relational systems, IEEE Trans. Pattern Anal. Mach. Intell. 13 (1991) 289–297.
[57] A. Lemos, W. Caminhas, F. Gomide, New uninorm-based neuron model and fuzzy neural networks, in: Fuzzy Information Processing Society (NAFIPS), 2010 Annual Meeting of the North American, IEEE, 2010, pp. 1–6..
[58] P. Souza, H. Ponce, E. Lughofer, Evolving fuzzy neural hydrocarbon networks: a model based on organic compounds, Knowl.-Based Syst. 203 (2020) 106099.
[59] E. Hüllermeier, K. Brinker, Learning valued preference structures for solving classification problems, Fuzzy Sets Syst. 159 (2008) 2337–2352.
[60] P. Angelov, Outside the box: an alternative data analytics framework, J. Autom. Mobile Robot. Intell. Syst. 8 (2014) 29–35.
[61] P. Angelov, X. Gu, D. Kangin, Empirical data analytics, Int. J. Intell. Syst. 32 (2017) 1261–1284.
[62] P. Angelov, X. Gu, D. Kangin, J. Principe, Empirical data analysis: a new tool for data analytics, in: 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2016, pp. 52–59..
[63] P.V. de Campos Souza, L.C.B. Torres, Regularized fuzzy neural network based on or neuron for time series forecasting, in: G.A. Barreto, R. Coelho (Eds.), Fuzzy Information Processing, Springer International Publishing, Cham, 2018, pp. 13–23.
[64] P.V.D.C. Souza, A.J. Guimaraes, V.S. Araujo, T.S. Rezende, V.J.S. Araujo, Regularized fuzzy neural networks to aid effort forecasting in the construction and software development, Int. J. Artif. Intell. Appl. 9 (2018) 13–26.

[65] A.J. Guimarães, P.V. de Campos Souza, E. Lughofer, Hybrid model for parkinson's disease prediction, in: M.-J. Lesot, S. Vieira, M.Z. Reformat, J.P. Carvalho, A. Wilbik, B. Bouchon-Meunier, R.R. Yager (Eds.), Information Processing and Management of Uncertainty in Knowledge-Based Systems, Springer International Publishing, Cham, 2020, pp. 621–634.
[66] E. Lughofer, On-line incremental feature weighting in evolving fuzzy classifiers, Fuzzy Sets Syst. 163 (2011) 1–23.
[67] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani, et al., Least angle regression, Ann. Stat. 32 (2004) 407–499.
[68] F.R. Bach, Bolasso: model consistent lasso estimation through the bootstrap, in: Proceedings of the 25th international conference on Machine learning, ACM, 2008, pp. 33–40..
[69] P.V.C. Souza, Regularized fuzzy neural networks for pattern classification problems, Int. J. Appl. Eng. Res. 13 (2018) 2985–2991.
[70] A. Albert, Regression and the Moore-Penrose pseudoinverse, Elsevier, 1972.
[71] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning: Data Mining, Inference and Prediction – Second Edition, Springer, New York Berlin Heidelberg, 2009.
[72] E. Lughofer, Flexible evolving fuzzy inference systems from data streams (FLEXFIS++), in: M. Sayed-Mouchaweh, E. Lughofer (Eds.), Learning in Non-Stationary Environments: Methods and Applications, Springer, New York, 2012, pp. 205–246.
[73] G.-B. Huang, L. Chen, C.K. Siew, et al., Universal approximation using incremental constructive feedforward networks with random hidden nodes, IEEE Trans. Neural Networks 17 (2006) 879–892.
[74] E. Lughofer, Evolving Fuzzy Systems—Methodologies, Advanced Concepts and Applications, Springer, Berlin Heidelberg, 2011.
[75] E. Lughofer, J.-L. Bouchot, A. Shaker, On-line elimination of local redundancies in evolving fuzzy systems, Evolving Syst. 2 (2011) 165–187.
[76] E.P. Klement, R. Mesiar, E. Pap, Triangular norms. position paper iii: continuous t-norms, Fuzzy Sets Syst. 145 (2004) 439–454.
[77] E. Soares, P. Angelov, X. Gu, Autonomous learning multiple-model zero-order classifier for heart sound classification, Appl. Soft Comput. 94 (2020) 106449.
[78] H. Ponce, P. de Campos Souza, A. Guimaraes, G. Gonzalez-Mora, Stochastic parallel extreme artificial hydrocarbon networks: an implementation for fast and robust supervised machine learning in high-dimensional data, Eng. Appl. Artif. Intell. 89 (2020) 103427.
[79] P.V. de Campos Souza, L.C.B. Torres, A.J. Guimaraes, V.S. Araujo, V.J.S. Araujo, T. S. Rezende, Data density-based clustering for regularized fuzzy neural networks based on nullneurons and robust activation function, Soft. Comput. 23 (2019) 12475–12489.
[80] K. Bache, M. Lichman, Uci machine learning repository, 2013..
[81] I.-C. Yeh, C.-H. Lien, The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients, Expert Syst. Appl. 36 (2009) 2473–2480.
[82] F. Khozeimeh, R. Alizadehsani, M. Roshanzamir, A. Khosravi, P. Layegh, S. Nahavandi, An expert system for selecting wart treatment method, Comput. Biol. Med. 81 (2017) 167–175.
[83] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, Smote: synthetic minority over-sampling technique, J. Artif. Intell. Res. 16 (2002) 321–357.
[84] K. Demertzis, L. Iliadis, A bio-inspired hybrid artificial intelligence framework for cyber security, Computation, Cryptography, and Network Security, Springer (2015) 161–193.
[85] X. Gu, P. Angelov, D. Kangin, J. Principe, Self-organised direction aware data partitioning algorithm, Inf. Sci. 423 (2018) 80–95.
[86] P.V.C. Souza, A.J. Guimaraes, V.S. Araujo, L.O. Batista, T.S. Rezende, An Interpretable Machine Learning Model for Human Fall Detection Systems Using Hybrid Intelligent Models, Springer International Publishing, Cham, 2020, pp. 181–205.
[87] L. Martínez-Villaseñor, H. Ponce, J. Brieva, E. Moya-Albor, J. Núñez-Martínez, C. Peñafort-Asturiano, Up-fall detection dataset: a multimodal approach, Sensors 19 (2019).



Paulo Vitor de Campos Souza Ph.D. student in Technical Sciences at Johannes Kepler University Linz - JKU-Austria. Master in Electrical Engineering from the Federal University of Minas Gerais (2015)- Brazil. Graduated in Information Systems from Centro Universitário UNA (2009) and Specialization in Informatics - Emphasis in Software Engineering from Universidade Federal de Minas Gerais (UFMG). He is an Information Technology-Development Analyst at the Federal Center for Technological Education of Minas Gerais - CEFET-MG. He worked as a professor of undergraduate courses at the UNA Faculty of Betim. His main research interest is focused on intelligent hybrid models, mainly Fuzzy neural networks. Data mining and binary pattern classification problems are also of interest.



Edwin Lughofer PhD in applied mathematics (Johannes Kepler University Linz -Linz, 2005). Key Researcher @ FLLL (Department of Knowledge-Based Mathematical Systems, Johannes Kepler University Linz). Awards: Best Paper at EFS 2006, Best Paper Finalist at GEFS 2008, Best Paper at MIM 2013, Best Paper at IEEE Intelligent Systems 2016 – Edited Book about "Predictive Maintenance in Dynamic Systems" with high-end research topics and modern industrial applications appeared in March 2019 (Springer New York). — Link to Amazon – Associate Editor of 'IEEE Transactions on Fuzzy Systems' (since 01.01.2020). – Member of the Editorial Board of 'Information Sciences' (since 01.01.2018). Specialized in: Evolving (fuzzy systems) systems, soft computing, machine learning, active learning, data mining (clustering), decision making, system identification, fault detection and identification, sensor fusion, computer vision.